Efficient $(1, s)$-Nucleus Decomposition on Billion-Scale Graphs

Anonymous Authors
Submission #XXX

Abstract

Extracting cohesive substructures from large graphs is a fundamental task in graph mining, with applications in community detection, dense-subgraph search, and role analysis. The $(1, s)$-*nucleus decomposition* is a higher-order generalization of k-core that ranks each vertex by the s-clique witnesses surrounding it (triangles for $s = 3$, 4-cliques for $s = 4$, and so on), giving sharper rankings than the standard k-core but at much higher cost. Existing studies compute $(1, s)$-nucleus inside a general (r, s) framework that encodes the witnesses with a Clique Path Index but maintains the index as mutable residual state during peeling. However, this mutability causes significant overhead in residual-index rewrites, memory consumption, and per-pop work, limiting scalability to large graphs and high s. We observe that the $(1, s)$ special case has a stronger invariant: vertex peeling preserves every edge among surviving vertices, so every CPI path stays structurally valid throughout the peel. This invariant turns exact decomposition into counter maintenance over a static CPI, which yields SPIN★ (a single-pass bucket-queue peel solver), ParaBuild (a parallel constructor for the shifted bottleneck), and BuildHier (a post-peel hierarchy emitter that reuses the preserved index). Experiments on 307 matched successful configurations show that SPIN★ achieves a 13.50× geometric-mean peel-time speedup and a 44.76× maximum peel-time speedup over the mutable baseline, while reducing memory usage by a 1.55× geometric-mean ratio; the static-state pipeline also scales to a billion-edge graph for which the mutable baseline completes only at s=2.

CCS Concepts

• **Theory of computation** → **Graph algorithms analysis**; • **Information systems** → *Data mining*.

Keywords

nucleus decomposition, clique counting, cohesive subgraphs, hierarchy

ACM Reference Format:
Anonymous Authors. 2027. Efficient $(1, s)$-Nucleus Decomposition on Billion-Scale Graphs. In *Proceedings of 2027 ACM SIGMOD International Conference on Management of Data (SIGMOD '27)*. ACM, New York, NY, USA, 16 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Core decomposition ranks vertices by the amount of cohesive structure that still surrounds them after weak vertices are removed [5, 41, 48] and supports a wide range of graph-mining tasks — community detection [13, 17, 18, 20], dense-subgraph discovery [19], influential-vertex and role analysis [11, 24, 30], network visualization [51], and applications in social, biological, and security networks [3, 33, 34]. The classical k-core uses edges as the witness, but many of these tasks need a stronger witness because a group may pass an edge-degree threshold while containing few triangles, 4-cliques, or larger coordinated patterns [1, 12, 22]. The $(1, s)$-*nucleus decomposition* [39, 40] gives this vertex-centric higher-order ranking: for a fixed s, it assigns each vertex the largest value k for which the vertex belongs to a maximal region where every vertex participates in at least k internal s-cliques. For $s = 2$ it reduces to the k-core; for larger s it ranks vertices by progressively tighter clique-supported cohesion.

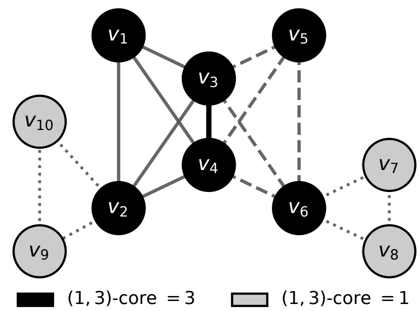

Figure 1: Running example of $(1, 3)$-nucleus decomposition.

Example 1.1. Figure 1 shows a 10-vertex graph at $s = 3$. The six vertices $v_1, \ldots, v_6$ belong to two triangle-rich K_4 blocks; each of them participates in at least three triangles supported by other vertices of the same value, so all six have $(1, 3)$-core value 3. The remaining four vertices $v_7, \ldots, v_{10}$ each lie in one of two pendant triangles ($\{v_6, v_7, v_8\}$ and $\{v_2, v_9, v_{10}\}$), supporting only one triangle each, so all four have $(1, 3)$-core value 1. A formal definition is given in Section 2; we revisit this graph as a running example throughout the paper.

Challenges. Computing $(1, s)$-nucleus decomposition at high s is hard because the number of s-cliques can be enormous; the current state of the art avoids explicit enumeration with the Clique Path Index, or CPI [49]. Using the CPI efficiently for the vertex-centric case $r = 1$ raises four challenges.

Residual state during peeling. The general (r, s) algorithm maintains the CPI as mutable residual state, storing per-path vertex sets, a reverse vertex-to-path index, and a priority queue, and rewriting these structures every time a vertex is popped. At $r = 1$, peeling removes vertices but never deletes edges, so it is not obvious that the same residual maintenance is necessary.

SIGMOD '27, June 2027, TBD

ACM ISBN 978-1-XXXX-XXXX-X/27/06…$15.00
https://doi.org/XXXXXXX.XXXXXXX

Memory footprint of the residual index. The residual structures dominate memory: per-path vertex hash sets, the reverse vertex-to-path map, and the live Bron–Kerbosch recursion tree together pay constant-factor overhead per vertex–path incidence on top of the clique-encoding cost. A layout whose memory grows with the vertex–path incidence count Σ alone would let the achievable s on a fixed-memory machine be dictated by Σ rather than by hash-table overhead.

Construction as the new bottleneck. Once peeling becomes counter-only, the static index builder takes seconds to minutes while peeling takes milliseconds, so construction is the next bottleneck and must be parallelised without paying the synchronisation cost of mutable path rewrites.

Hierarchy emission from a mutating index. The clique-coherent nucleus hierarchy of Definition 2.3 witnesses s-connectivity through s-cliques contained in each super-level set, so emitting it requires access to the s-cliques themselves. If the CPI is mutated during peeling, those cliques are no longer available afterwards and the hierarchy needs either a second clique-enumeration pass or per-peel-level state, both of which can dominate total cost.

State-of-the-Art and Limitations. The current state of the art for general (r, s)-nucleus decomposition is CND [49], which maintains the CPI as mutable residual state and inherits four weaknesses when restricted to $r = 1$.

Mutable residual-index maintenance. CND rewrites path records and updates a hash-indexed reverse map at every peel step. These rewrites are a real cost rather than a constant factor: SPIN*, which replaces them with closed-form counter updates over a static index, delivers a 13.50× geometric-mean peel-time speedup over CND on 307 matched cells (Section 8).

High memory consumption. The residual structures that CND must keep mutable in the general (r, s) setting — per-path membership queries, reverse vertex-to-path lookup, and the live recursion tree used to rewrite paths — together cost roughly 88Σ bytes per incidence (Section 8, Table 3), close to an order of magnitude more than what a static encoding of the same witnesses requires. The geometric-mean memory ratio CND/SPIN* is 1.55× on the matched cells and 3.92× at maximum; on a billion-scale graph CND completes only at s=2 (using 400 GB of 503 GB) and exceeds the budget for every $s \geq 3$, while SPIN* finishes the full s sweep.

Construction as the residual bottleneck. Even after cheap peeling, CND pays the full construction cost; on `web-it-2004` (our highest CPI-build-cost benchmark) at $s = 3$, construction takes about 137 seconds and peeling takes about 59 milliseconds, so the static index builder dominates total runtime by more than 2,000× on this configuration. Without a parallel builder, the end-to-end speedup over CND stays at 1.10× on the matched cells even though the peel-time speedup is 13.50×.

Hierarchy emission re-enumerates or replicates. Because CND edits the CPI during peeling, its s-clique witnesses are gone when the hierarchy is requested. The natural workaround — the general (r, s) level-DSU hierarchy specialised to r=1 — maintains one disjoint-set forest per core level during the peel and exceeds the available 503 GB on `wiki-Talk` at $s \geq 9$ where our pipeline finishes (Figure 9); even where it fits, it uses up to 43× more memory (gmean 13× over 24 matched cells) than emitting the same hierarchy from a preserved CPI.

Our Solution and Contributions. For $(1, s)$-nucleus decomposition we exploit a static-CPI invariant (Theorem 4.1) to deliver a single pipeline that addresses all four challenges above.

*Static-*CPI *invariant for $(1, s)$.* We prove that vertex peeling preserves every CPI path (Theorem 4.1) and realize the resulting counter-only peel as SPIN* (Algorithm 4), which drops per-incidence space from $\approx 88\Sigma$ bytes to $\approx 8\Sigma$ bytes (Table 3). On 307 matched cells, SPIN* achieves a 13.50× geometric-mean peel-time speedup and a 1.55× geometric-mean memory reduction over CND, and on a billion-scale graph SPIN* decomposes the full s sweep while CND OOMs for every $s \geq 3$.

Parallel CPI *construction.* ParaBuild (Algorithm 5) parallelises CPI construction across seed vertices, removing the construction bottleneck at $r = 1$; empirical scaling reaches 23.4× at T=24 threads.

Core-value hierarchy from the static CPI. BuildHier (Algorithm 6) emits the s-clique-connected super-level join tree from the preserved CPI in $O(\Sigma \log \Sigma)$ time without re-enumerating any s-clique, using up to 43× less memory (gmean 13× over 24 matched cells) than the level-DSU baseline.

Comprehensive experimental evaluation. We compare against CND on ten real-world graphs (307 matched configurations, headline peel speedup 13.50× gmean / 44.76× max, memory-usage ratio 1.55× gmean), report phase behaviour and parallel construction, push the pipeline to a billion-scale graph (where CND only completes at s=2), and use case studies to evaluate whether the $r = 1$ specialization is practically useful.

2 Preliminaries

Graph notation. We work with a simple undirected graph $G = (V, E)$. For a vertex v, $N_G(v) = \{u \in V \mid (u, v) \in E\}$ is its neighbor set and $\deg_G(v) = |N_G(v)|$ is its degree. For $X \subseteq V$, $G[X]$ is the subgraph induced by X. The implementation stores G in CSR form with an `offsets` array and an `edges` array.

Definition 2.1 (k-Clique)**.** A k*-clique* is a vertex set $C \subseteq V$ with $|C| = k$ such that every two distinct vertices in C are adjacent.

Definition 2.2 (s-Clique Support)**.** Given $s \geq 2$, the s*-clique support* of a vertex v in G is

$$\deg^{(s)}(v) \;=\; \big|\{S \subseteq V \mid v \in S,\ |S| = s,\ S \text{ is a clique in } G\}\big|.$$

$\deg^{(s)}(v)$ is evaluated inside the current induced subgraph when a peel level is being defined; the subscript is dropped when the graph is clear.

Definition 2.3 (k-$(1, s)$-Nucleus)**.** Let $X \subseteq V$. The induced subgraph $G[X]$ is a k-$(1, s)$*-nucleus* if:

(1) every vertex in X belongs to at least k s-cliques contained in $G[X]$;
(2) $G[X]$ is s*-connected*: for any two vertices $u, v \in X$, there is a sequence $u = v_1, \ldots, v_t = v$ such that each pair $\{v_i, v_{i+1}\}$ is contained in an s-clique of $G[X]$;
(3) $G[X]$ is maximal with respect to adding vertices while preserving the first two conditions.

Algorithm 1: BuildCPI(G, s) — standard CPI construction [49].

Input: Undirected graph $G = (V, E)$, clique size s.
Output: Clique Path Index $\mathcal{P}$ of G for clique size s.

1 Compute a degeneracy ordering $(v_1, \ldots, v_n)$ of G and orient every edge from lower to higher rank, giving $N^+(\cdot)$;
2 $\mathcal{P} \leftarrow \emptyset$;
3 **foreach** $i \leftarrow 1$ **to** n **do**
4   ExpandPath($N^+(v_i)$, $\{v_i\}$, $\emptyset$) ; // one root per vertex
5 **return** $\mathcal{P}$ **Function** ExpandPath(P, V_h, V_p):
6   **if** $|V_h| > s$ **then return**;
7   **if** $P = \emptyset$ **then**
8   **if** $|V_h| + |V_p| \geq s$ **then** append the path (V_h, V_p) to $\mathcal{P}$;
9   **return**
10   $u_{\text{piv}} \leftarrow \arg\max_{x \in P} |P \cap N^+(x)|$; // Tomita pivot
11   **foreach** $v \in P \setminus N(u_{\text{piv}})$ **do**
12     $P \leftarrow P \setminus \{v\}$;
13   **if** $v = u_{\text{piv}}$ **then**
14   ExpandPath($P \cap N(v)$, V_h, $V_p \cup \{v\}$);
15   **else**
16   ExpandPath($P \cap N(v)$, $V_h \cup \{v\}$, V_p);

Definition 2.4 (Core Number). Given $s \geq 2$, the $(1, s)$*-nucleus core number* of a vertex v, denoted $\kappa_s(v)$, is the largest k for which v belongs to some k-$(1, s)$-nucleus of G.

The Clique Path Index. The Clique Path Index, or CPI, compactly represents s-cliques without listing them one by one [49].

Definition 2.5 (CPI Path). A CPI *path* is a pair $P = (V_h(P), V_p(P))$ of disjoint vertex sets such that $V_h(P) \cup V_p(P)$ is a clique in G and $|V_h(P)| \leq s \leq |V_h(P)| + |V_p(P)|$. The set $V_h(P)$ is the *hold set*: every encoded s-clique contains all of these vertices. The set $V_p(P)$ is the *pivot set*: an encoded s-clique chooses vertices from this pool. The *pivot requirement* of P is $\eta_P = s - |V_h(P)|$, the number of pivot vertices needed to complete the hold set into an s-clique. The boundary case $\eta_P = 0$ (i.e. $|V_h(P)| = s$) encodes exactly one s-clique — the hold set itself — and we adopt the standard convention $\binom{m}{-1} = 0$ so the support-loss formula of Lemma 4.2 returns 0 for any pivot-removal event on such a path. The set of s-cliques encoded by P is

$$C(P) = \left\{ V_h(P) \cup P' \mid P' \subseteq V_p(P),\ |P'| = \eta_P \right\}.$$

Its cardinality is $\binom{|V_P(P)|}{\eta_P}$.

The CPI is a set $\mathcal{P}$ of paths such that every s-clique is encoded by exactly one path. We write

$$\Sigma = \sum_{P \in \mathcal{P}} \left(|V_h(P)| + |V_p(P)| \right)$$

for the vertex–path incidence count. We also write $\kappa_{\max} = \max_v \deg^{(s)}(v)$. The CPI is built by a degeneracy-ordered Bron–Kerbosch search with Tomita pivoting [9, 15, 44, 49]; we reproduce the construction here as Algorithm 1 because the rest of the paper depends on its leaf shape.

Algorithm 1 computes a degeneracy ordering of G and orients edges from lower to higher rank so each s-clique has a unique smallest-rank owner (Line 1); for every vertex v_i, ExpandPath is invoked with hold set $\{v_i\}$ and forward-neighbour candidate set $N^+(v_i)$ (Lines 3-4). A recursive call returns immediately when the hold set already exceeds s (Line 6), and emits a path when the candidate set is empty and the hold-plus-pivot size meets s (Lines 8-9). Otherwise the Tomita pivot u_{piv} is the vertex of P with the most forward neighbours inside P (Line 10), and every non-neighbour of u_{piv} is recursively expanded: branching on the pivot itself extends the pivot set, branching on any other vertex extends the hold set (Lines 11-16).

Reading clique counts off the CPI. Once the CPI is built, each vertex's exact s-clique support is a sum over the paths through it: every s-clique through v is encoded by exactly one path with v either in the hold set or in the pivot set, and the count in each case is a single binomial in the path's pivot size.

Lemma 2.6 (Support Formula [49]). *For every $v \in V$,*

$$\deg^{(s)}(v) = \sum_{P : v \in V_h(P)} \binom{|V_p(P)|}{\eta_P} + \sum_{P : v \in V_p(P)} \binom{|V_p(P)| - 1}{\eta_P - 1}.$$

The first sum is over paths with v in the hold: every choice of η_P pivots from $V_p(P)$ completes $V_h(P)$ into a distinct s-clique through v, giving $\binom{|V_P(P)|}{\eta_P}$ cliques per path. The second sum is over paths with v in the pivot set: v fills one of the η_P pivot slots, and the remaining $\eta_P - 1$ are chosen from the other $|V_p(P)| - 1$ pivots, giving $\binom{|V_P(P)|-1}{\eta_P-1}$ cliques per path. Every $\deg^{(s)}(v)$ in the rest of the paper is evaluated through this formula, never by listing cliques.
Problem statement. Given G and $s \geq 2$, compute $\kappa_s(v)$ for every $v \in V$; the nucleus hierarchy is constructed afterwards (Section 7).

3 The Mutable-CPI Baseline

CND [49] is the state-of-the-art exact algorithm for general (r, s)-nucleus decomposition. The remainder of this section walks through its peeling loop, the residual structures it maintains, and the operations this paper specialises away when restricted to $r = 1$.

The peeling loop. CND follows the standard core-decomposition template [5]: it maintains the residual support of every alive r-clique in a priority queue, repeatedly pops the r-clique of minimum residual support, assigns it the current peel level as its core value, and propagates the support loss to the remaining alive r-cliques. At $r = 1$ the queue items are vertices, so each iteration pops a vertex v, records $\kappa_s(v)$, and decrements the residual support of every vertex that loses an s-clique witness because of v's removal. The non-trivial work happens during this support-loss propagation, because the CPI that encodes the witnesses must be brought into a state consistent with v being gone before the next pop.

Residual state. To keep the witness counts consistent across peels, CND maintains the CPI as mutable residual state. For each path P it stores the hold and pivot vertex sets as hash sets so that the test "does v belong to this path?" is constant time. A separate reverse map sends every vertex u to the list of paths that contain it, so that when u is popped the algorithm can find the affected paths without scanning all of $\mathcal{P}$. A heap (or bucket queue) keeps every alive vertex ordered by current residual support, with decrease-key operations for the support drops triggered at each peel event. Finally, the Bron–Kerbosch recursion tree that produced the CPI is kept mutable: when a peel event invalidates a path, the algorithm walks into the tree and rewrites or removes the affected nodes.

Per-peel work. A single peel of vertex v costs:

- a reverse-map lookup of v's path list, then for every such path P a constant-time hash-set query to determine whether v is a hold or a pivot;
- a path rewrite: if v is a hold, the path is deleted from the recursion tree and its reverse-map entries for all other vertices in $V_h(P) \cup V_p(P)$ are erased; if v is a pivot, the path's pivot set shrinks by one and the reverse-map entry for v on this path is erased;
- a support recompute for every path-mate u using the encoded s-clique family that survives the rewrite, followed by a decrease-key on u in the heap.

The dominant components are the path rewrites and the reverse-map erases; together they scale with the number of vertex–path incidences touched by the event, which is what gives CND its observed cost profile.

Why this state is needed in the general case. The mutable design is not over-engineering for the general (r, s) setting. When the peeled object is an edge ($r = 2$) or a higher-order clique ($r \geq 3$), the removal can destroy edges among vertices that remain alive; some of the cliques that a path encodes therefore cease to exist as cliques. In that setting, the stored hold and pivot sets of P no longer define a valid encoded clique family, so the path must be split, shrunk, or deleted by walking back into the recursion tree. The reverse map and the per-path hash sets exist precisely to support that surgery efficiently.

Why this state is wasteful at $r = 1$**.** The $(1, s)$ case has a stronger invariant: vertex peeling removes vertices but does not delete edges, so a set of vertices that formed a clique when the CPI was built remains a clique throughout the peel (we prove this in Section 4). A path's encoded clique family therefore stays structurally valid; only the question of whether a particular vertex of $V_h(P) \cup V_p(P)$ is still alive changes. This means the path rewrites, the reverse-map erases, and the recursion-tree surgery in the per-peel cost above are all unnecessary at $r = 1$ — they maintain a mutable residual state that is logically equivalent to a static incidence layout plus a single integer counter per path. The next four sections cash in this observation: Section 4 states the static-CPI invariant in counter form; Section 5 derives SPIN and SPIN* over the resulting static index; Section 6 parallelises construction once peeling becomes counter-only; and Section 7 extracts the nucleus hierarchy from the preserved CPI without any clique re-enumeration.

4 Static CPI for $r = 1$

We prove the central invariant of the paper: at $r = 1$, vertex peeling never deletes an edge among the surviving vertices, so the stored clique $V_h(P) \cup V_p(P)$ of every path remains structurally valid throughout the peel and the residual state that CND maintains (Section 3) is unnecessary.

Static structure and dynamic counters. Recall from Section 2 that a path stores the hold set $V_h(P)$, the pivot set $V_p(P)$, and the pivot requirement $\eta_P = s - |V_h(P)|$; during peeling none of these three objects is edited. The dynamic state is only a liveness bit and an integer p_P, the number of pivots in $V_p(P)$ that have not yet been peeled. If a peeled vertex is a hold, every encoded s-clique contains that vertex, so the path contributes nothing after the event; if a peeled vertex is a pivot, the path remains the same symbolic object and p_P decreases by one. The next theorem records this counter form: a peel event either kills a path contribution or reduces an effective pivot count.

Theorem 4.1 (Vertex Removal on CPI, Counter Form)**.** *Let P be a path of the* CPI *of G, and let $v \in V$ be removed from G.*

(1) If $v \in V_h(P)$, P encodes no surviving s-clique.

(2) If $v \in V_p(P)$, the surviving s-cliques are encoded by the same $V_h(P)$ together with $V_p(P) \setminus \{v\}$. Their count depends only on $|V_p(P)|$. A counter $p_P = |V_p(P)|$ decremented by one is therefore support-equivalent to deleting v from $V_p(P)$. If $p_P - 1 < \eta_P$, no surviving s-clique is encoded.

(3) Otherwise, P is unaffected.

The dynamic state of P during peeling reduces to an integer counter p_P, the fixed pivot requirement η_P, and a liveness bit. The stored sets $V_h(P)$ and $V_p(P)$ are never modified.

Proof. P encodes $C(P) = \{V_h(P) \cup P' \mid P' \subseteq V_p(P),\ |P'| = \eta_P\}$. If $v \in V_h(P)$, every encoded clique contains v and none survives. If $v \in V_p(P)$, the surviving cliques are $\{V_h(P) \cup P' \mid P' \subseteq V_p(P) \setminus \{v\},\ |P'| = \eta_P\}$, a family of size $\binom{|V_p(P)|-1}{\eta_P}$ that depends on $V_p(P)$ only through its cardinality; by Lemma 2.6 every support contribution of the path depends only on this cardinality, so the counter update $p_P \leftarrow p_P - 1$ is support-equivalent to physical deletion, and the family is empty iff $p_P - 1 < \eta_P$. If $v \notin V_h(P) \cup V_p(P)$, no encoded clique mentions v and the family is unchanged. □ □

Boundary of the invariant. The theorem is specific to $r = 1$. When $r \geq 2$, a peel event removes an edge or a higher-order clique from the residual object. That event can break the clique represented by $V_h(P) \cup V_p(P)$. In that setting, a path may need to be split, shrunk, or rebuilt. The mutable CPI machinery of [49] remains the appropriate general tool, and this paper does not subsume it.

Support loss from one path event. Theorem 4.1 leaves only cardinality changes. The support lost by any other live vertex on the path is therefore a binomial difference.

Lemma 4.2 (Support Loss for a Path Event)**.** *Let P be a live path with pivot count $p = |V_p(P)| \geq \eta$, where $\eta = \eta_P$ is the pivot requirement, and let an event remove $d \geq 1$ live pivots from P. For every other alive vertex u incident to P:*

$$\Delta_{hold}(p, \eta, d) = \binom{p}{\eta} - \binom{p-d}{\eta} \qquad \text{if } u \in V_h(P),$$

$$\Delta_{pivot}(p, \eta, d) = \binom{p-1}{\eta-1} - \binom{p-d-1}{\eta-1} \qquad \text{if } u \in V_p(P).$$

A path-killing event, caused by a hold peel or by $p - d < \eta$, is the limiting case in which the entire contribution vanishes: $\binom{p}{\eta}$ for surviving holds and $\binom{p-1}{\eta-1}$ for surviving pivots.

Proof. By Lemma 2.6, a hold u contributes $\binom{p}{\eta}$ before the event and $\binom{p-d}{\eta}$ after; a pivot u occupies one slot and contributes $\binom{p-1}{\eta-1}$ before and $\binom{p-d-1}{\eta-1}$ after. Subtracting after from before gives the two formulas. □ □

Running example. Figure 1 shows the running example at $s = 3$: two K_4 blocks share edge (v_3, v_4), and two pendant triangles attach at v_2 and v_6. The six CPI paths in Table 1 encode the ten triangle witnesses through stored V_h, V_p sets and the binomial count

Table 1: CPI paths for the running example at $s = 3$.

Path	V_h	V_p	$\eta = s - \lvert V_h \rvert$	$\binom{\lvert V_p \rvert}{\eta}$
P_1	$\{v_7\}$	$\{v_6, v_8\}$	2	1
P_2	$\{v_9\}$	$\{v_2, v_{10}\}$	2	1
P_3	$\{v_1\}$	$\{v_2, v_3, v_4\}$	2	3
P_4	$\{v_2\}$	$\{v_3, v_4\}$	2	1
P_5	$\{v_3\}$	$\{v_4, v_5, v_6\}$	2	3
P_6	$\{v_4\}$	$\{v_5, v_6\}$	2	1
Total				**10**

$\binom{|V_p|}{s-|V_h|}$. The $(1, 3)$-core values fall into two bands: $\kappa_3(v_i) = 3$ for $i \in \{1, \dots, 6\}$ (the two K_4 blocks) and $\kappa_3(v_i) = 1$ for $i \in \{7, \dots, 10\}$ (the pendant triangles); this is the partition shown by the node shading in Figure 1. The example exercises both events of Theorem 4.1: a path dies when a hold vertex is removed, and a path's pivot counter shrinks when a pivot vertex is removed.

Example 4.3 (Peel event walk-through). Consider the path $P_3 = (\{v_1\}, \{v_2, v_3, v_4\})$ in Table 1, with $p = 3$ and $\eta = 2$. Peeling v_2 as a pivot triggers Lemma 4.2 with $d = 1$. The surviving hold v_1 loses $\binom{3}{2} - \binom{2}{2} = 2$ from its support. The surviving pivots v_3 and v_4 each lose $\binom{2}{1} - \binom{1}{1} = 1$. The stored set $V_p(P_3)$ is not modified. Only p_{P_3} changes from 3 to 2.

5 Computing Core Values over Static CPI

The static-CPI invariant of Section 4 becomes a concrete algorithm for κ_s through a one-vertex update rule built directly on clique-witness counts (Section 5.1, Algorithm 2). Two solvers share the same update: SPIN applies it by repeated full passes (Section 5.2, Algorithm 3); SPIN* schedules it in peel order so unaffected vertices are not revisited (Section 5.3, Algorithm 4), and Lemma 5.5 proves the two reach the same fixed point.

5.1 Vertex updates from static witnesses

The static-CPI invariant counts witnesses without rebuilding paths; for core values we additionally need a rule for lowering a vertex value from the witnesses that remain. At level k, the vertex needs at least k encoded clique witnesses whose other vertices can also sustain level k. For a tentative value vector, define the one-vertex update

$$\Phi_h(v) = \max\Big\{k \geq 0 \,\Big|\, \sum_{P \ni v} W_v(P, h, k) \geq k\Big\}.$$

Here $W_v(P, h, k)$ is the number of encoded s-cliques on path P through v whose other vertices have value at least k, and the sum runs over all paths through v. For clique size two, the witnesses are edges, so the update becomes the standard core fixed-point test over neighbors [21, 31]. For larger clique sizes, the counted items are clique witnesses, not distinct neighbor vertices. The next lemma counts $\sum_{P \ni v} W_v(P, h, k)$ from CPI paths without enumerating those witnesses.

Lemma 5.1 (Per-path Witness Count). *Fix a vertex v, a current vector h, and a threshold $k \geq 1$. For a path P with $v \in V_h(P) \cup V_p(P)$, let $H_k(P)$ and $Q_k(P)$ be the number of vertices in $V_h(P) \setminus \{v\}$ and $V_p(P) \setminus \{v\}$, respectively, whose current h-value is at least k. The number $W_v(P, h, k)$ of encoded s-cliques through v in which every other vertex has h-value at least k is:*

(1) if $H_k(P) < |V_h(P) \setminus \{v\}|$, then $W_v(P, h, k) = 0$;

(2) if $v \in V_h(P)$, then $W_v(P, h, k) = \binom{Q_k(P)}{\eta_P}$;

(3) if $v \in V_p(P)$, then $W_v(P, h, k) = \binom{Q_k(P)}{\eta_P - 1}$.

Summing over the paths through v gives the total threshold count $\sum_{P \ni v} W_v(P, h, k)$, and $\Phi_h(v)$ is the largest k for which this sum is at least k.

Algorithm 2: VertexSupport(v, h) — one-vertex update from static clique witnesses.

Input: Vertex $v \in V$; value array h with $h[u]$ the running estimate of $\kappa_s(u)$ for each $u \in V$; CPI $\mathcal{P}$.
Output: The threshold-support value $\Phi_h(v)$.

```
1 C[0,...,h[v]] ← 0 ;                          // threshold witness counts
2 foreach path P ∈ 𝒫 through v do
3     for k ← 1 to h[v] do
4         C[k] ← C[k] + W_v(P, h, k);                        // Lemma 5.1
5 for k ← h[v] to 1 do
6     if C[k] ≥ k then return k;
7 return 0
```

Proof. Each encoded s-clique through v has the form $V_h(P) \cup P'$ with $P' \subseteq V_p(P)$ and $|P'| = \eta_P$. If any other hold vertex fails threshold k, no such clique qualifies; otherwise, $v \in V_h(P)$ leaves all η_P pivot slots to be filled from the $Q_k(P)$ passing pivots and gives $\binom{Q_k(P)}{\eta_P}$ qualifying cliques, while $v \in V_p(P)$ leaves $\eta_P - 1$ slots to fill and gives $\binom{Q_k(P)}{\eta_P - 1}$. Summing over the paths through v gives the stated total. $\square$ $\square$

VertexSupport(v, h) computes this update for one vertex. During one call, the algorithm builds only local threshold state for v. After any prefix of the paths through v, that local state contains exactly the witnesses contributed by the scanned paths. Scanning the next path adds the binomial counts from Lemma 5.1, and the largest threshold with enough witnesses is the updated value. Because the call reads the static CPI but never mutates a path, the same witness rule can be used by both solvers below.

Algorithm 2 zeroes the local threshold-count array up to $h[v]$ (Line 1), accumulates the per-threshold witness count $W_v(P, h, k)$ into $C[k]$ for every path through v using the closed form of Lemma 5.1 (Lines 2-4), and returns the largest k at which $C[k] \geq k$ by scanning C from $h[v]$ downward (Lines 5-7); otherwise it returns 0. The pseudocode is shown in its semantic form for clarity; the implementation never materialises a dense $C[0, \dots, h[v]]$ array (which would be infeasible when $h[v]$ approaches $\deg^{(s)}(v)$, easily 10^{10}+ on dense graphs) but instead replaces the inner k-loop with a single pass per scanned path that sorts the path's vertex values once and computes $W_v(P, h, \cdot)$ at every threshold via prefix sums. Let $W_v = \sum_{P \ni v}(|V_h(P)| + |V_p(P)|)$ denote the total incidence weight at v and $L_{\max} = \max_P(|V_h(P)| + |V_p(P)|)$ the maximum path size. Per-call cost is $O(W_v \log L_{\max})$. The call reads the static CPI and the vector h, and it mutates no path.

5.2 Direct Static-Path Iteration

SPIN is the most literal solver of the threshold-support equation: it scans all vertices, computes a next value for each vertex from the previous value vector via VertexSupport, and repeats until a full scan makes no change. The only mutable data are the current and next value vectors; the current vector starts at the initial s-clique support, so every entry is an upper bound on the final core value, and each pass can only lower entries. A fixed point means every

Algorithm 3: SPIN — direct static-path fixed-point iteration.

```
   Input: Graph G, clique size s, CPI 𝒫.
   Output: κ_s(v) for every v ∈ V.
1  foreach v ∈ V do
2  └  h[v] ← deg^(s)(v) ;                  // closed form from Lemma 2.6
3  changed ← true;
4  while changed do
5  │  changed ← false;   h_new ← h;
6  │  foreach v ∈ V in a fixed order do
7  │  │  h_new[v] ← VertexSupport(v, h);
8  │  └  if h_new[v] ≠ h[v] then changed ← true;
9  └  h ← h_new;
10 foreach v ∈ V do
11 └  κ_s(v) ← h[v];
12 return κ_s
```

vertex value is consistent with the witnesses whose other vertices have high enough current value, which makes SPIN the reference formulation for the static-CPI computation even though repeated full scans can be expensive.

Algorithm 3 initialises $h[v]$ to the initial s-clique support $\deg^{(s)}(v)$ for every vertex via the closed form of Lemma 2.6 (Lines 1-2). The main loop (Lines 3-9) iterates until no entry changes in a full pass: it sets a *changed* flag to false (Line 5), copies the current vector into h_{new} (Line 5), then calls VertexSupport(v, h) for every vertex in a fixed order and stores the result into $h_{\text{new}}[v]$ (Lines 6-7); if any update lowered an entry the flag is set, otherwise the pass exits. The swap $h \leftarrow h_{\text{new}}$ at the end of the pass (Line 9) avoids order dependence inside the pass. The final core values are read off the converged vector (Lines 10-11) and returned (Line 12). A pass with no changed entry is the certificate that the fixed point has been reached.

Example 5.2 (Static-path update on the running example). Initialise $h[v] \leftarrow \deg^{(s)}(v)$ using Lemma 2.6; on the running example $h[v_4] = 6$ (contributions 2+1+2 from P_3, P_4, P_5 as pivot and 1 from P_6 as hold; see Table 1). The first call VertexSupport(v_4, h) scans the four paths through v_4 and counts threshold witnesses via Lemma 5.1. At threshold $k = 3$, every other vertex on each of the four paths has $h \geq 3$, so the witness sums are 2+1+2+1 = 6; the largest k with $\sum_{P \ni v_4} W_{v_4}(P, h, k) \geq k$ is $\Phi_h(v_4) = 3$, the final $\kappa_3(v_4)$. SPIN reaches the fixed point in a small number of full passes, but each pass scans every vertex regardless of whether its value can still change; this is the redundancy that SPIN* eliminates in Section 5.3.

We next establish that this iteration terminates and that the fixed point it reaches is the $(1, s)$-nucleus core number of Definition 2.4.

Lemma 5.3 (Fixed Point Correctness). *Initialised at $h^{(0)}[v] = \deg^{(s)}(v)$, Algorithm 3 converges in at most $n + 1$ passes to $h[v] = \kappa_s(v)$ for every $v \in V$, and the sequence $h^{(t)}[v]$ is monotonically non-increasing.*

Proof. By the operator definition, $h^{(t+1)}[v] \geq k$ iff v has at least k incident s-cliques whose other vertices all satisfy $h^{(t)} \geq k$, so $\deg^{(s)}(v)$ is the largest possible value and the sequence is monotone non-increasing on the non-negative integers. Once a vertex's value equals its final $\kappa_s(v)$, the operator never lowers it further (by the converse direction below). Each pass that is not already a fixed point lowers the value of at least one not-yet-finalised vertex, so at most n such passes occur before the fixed point $h^\star$ is reached and one more pass certifies it. At $h^\star$, let $S_k = \{v \mid h^\star[v] \geq k\}$; every $v \in S_k$ has at least k incident s-cliques entirely inside S_k, and each such clique lies in v's s-connected component $C \subseteq S_k$ (because the clique itself is the connecting witness), so C satisfies Conditions (i)–(iii) of Definition 2.3 and is a k-$(1, s)$-nucleus, giving $\kappa_s(v) \geq k$. Conversely, if v lies in a k-$(1, s)$-nucleus C', we show by induction on t that $h^{(t)}[u] \geq k$ for every $u \in C'$. The base case $t = 0$ holds since each $u \in C'$ is in at least k s-cliques inside C' (by Definition 2.3), so $\deg^{(s)}(u) \geq k = h^{(0)}[u]$. For the inductive step, assume $h^{(t)}[u'] \geq k$ for every $u' \in C'$; then u has at least k s-cliques inside C' whose other vertices all satisfy $h^{(t)} \geq k$, so the threshold-support operator yields $h^{(t+1)}[u] \geq k$. Therefore $h^\star[v] \geq k$. Hence $h^\star[v] = \kappa_s(v)$. □ □

The cost is the per-pass work (one VertexSupport call per vertex) times the number of passes:

Theorem 5.4 (SPIN Complexity). *Algorithm 3 computes κ_s in $O(T \cdot \Sigma \cdot L_{\max} \cdot \log L_{\max})$ work, where $T \leq n + 1$ is the number of passes (Lemma 5.3) and $L_{\max}$ is the maximum path size (Section 5.1).*

Proof. Each pass calls VertexSupport(v, h) for every $v \in V$ at per-call cost $O(W_v \log L_{\max})$ (Section 5.1). Summing, $\sum_v W_v = \sum_P |P|^2 \leq L_{\max} \cdot \Sigma$, so a pass costs $O(\Sigma \cdot L_{\max} \cdot \log L_{\max})$. Lemma 5.3 bounds the number of passes by $n + 1$. □ □

5.3 Eliminating Redundant Sweeps

SPIN is useful because it exposes the equations, but its full passes recompute many vertices whose values can no longer change. Once a vertex reaches its final peel level, later events cannot increase it. SPIN* removes this redundancy by using the same nondecreasing peel order as classical core decomposition. The algorithm keeps the same value semantics as SPIN, but it updates them with local path events instead of global passes. For each path, it stores whether the path is still live, how many pivot vertices have not been finalized, and the fixed pivot requirement. It also keeps a finalized set and a bucket queue keyed by the current value. When one minimum vertex is popped, only paths containing that vertex can change. Theorem 4.1 says how each such path changes, and Lemma 4.2 gives the exact support loss for the unfinalized path-mates. Thus the invariant is local: unfinalized vertex values reflect the current live path-counter contributions, clipped below by the current peel level.

SPIN* is an optimization of SPIN, not a different decomposition objective. It keeps the same witness equation but changes the schedule. Instead of scanning every vertex after every pass, it updates only the vertices whose path contributions changed. Algorithm 4 outputs the core values and emits a sorted deletion log as a byproduct. Its running time depends on U, the number of incremental support-update events that actually occur, rather than on the number of full scans.

Algorithm 4 first initialises every path's pivot count p_P, pivot requirement η_P, and liveness bit (Lines 1-2), then sets each vertex's working value $h[v]$ to its initial s-clique support and inserts v into the corresponding bucket (Lines 3-4). The main loop (Lines 6-19) advances the peel level k to the smallest non-empty bucket and pops the next vertex v (Line 7), finalises $\kappa_s(v) \leftarrow k$ (Line 8), and walks

Algorithm 4: SPIN* — core values via single-pass bucket-queue peeling.

Input: Graph G, clique size s, CPI $\mathcal{P}$.
Output: $\kappa_s(v)$ for every $v \in V$.

1 **foreach** *path* $P \in \mathcal{P}$ **do**
2 $\quad p_P \leftarrow |V_p(P)|$; $\eta_P \leftarrow s - |V_h(P)|$; $\text{alive}_P \leftarrow (p_P \geq \eta_P)$;
3 **foreach** $v \in V$ **do**
4 $\quad h[v] \leftarrow \deg^{(s)}(v)$; insert v into bucket $B[h[v]]$;
5 $k \leftarrow 0$; $V_{\text{fin}} \leftarrow \emptyset$;
6 **while** $|V_{\text{fin}}| < n$ **do**
7 $\quad$ advance k until $B[k] \neq \emptyset$; pop v from $B[k]$;
8 $\quad \kappa_s(v) \leftarrow k$; append (v, k) to the deletion log; $V_{\text{fin}} \leftarrow V_{\text{fin}} \cup \{v\}$;
9 $\quad \text{Aff} \leftarrow \emptyset$;
10 $\quad$ **foreach** P *with* alive_P *and* $v \in V_h(P) \cup V_p(P)$ **do**
11 $\quad\quad \text{Aff} \leftarrow \text{Aff} \cup ((V_h(P) \cup V_p(P)) \setminus \{v\})$;
12 $\quad\quad$ **if** $v \in V_h(P)$ **then**
13 $\quad\quad\quad \text{alive}_P \leftarrow$ false; // path death
14 $\quad\quad$ **else**
15 $\quad\quad\quad p_P \leftarrow p_P - 1$;
16 $\quad\quad\quad$ **if** $p_P < \eta_P$ **then** $\text{alive}_P \leftarrow$ false;
17 $\quad$ **foreach** $u \in \text{Aff} \setminus V_{\text{fin}}$ **do**
18 $\quad\quad y \leftarrow \max\{k, \text{VertexSupport}(u, h)\}$; // implemented incrementally with Lemma 4.2
19 $\quad\quad$ **if** $y < h[u]$ **then** move u from $B[h[u]]$ to $B[y]$; $h[u] \leftarrow y$;
20 **return** κ_s

every live path that contains v (Lines 10-16): if v is a hold, the path dies (Line 13); if v is a pivot, the path's pivot counter decrements and the path also dies once $p_P < \eta_P$ (Lines 15-16). Affected path-mates that have not been finalised are recomputed via Lemma 4.2 (Line 18) and moved to a lower bucket if their value dropped (Line 19). The pseudocode writes the affected update as VertexSupport to keep the fixed-point semantics visible; the implementation evaluates the changed contribution with the binomial loss formula rather than recomputing from scratch, so the total work scales with the number of incremental update events U rather than with the number of full scans. Buckets maintain the minimum-h vertex because values only decrease and are bounded by $\kappa_{\max}$ [5].

The schedule is different from SPIN, but the fixed point it reaches is the same:

Lemma 5.5 (SPIN* Equivalence)**.** *Algorithm 4 returns the same κ_s values as Algorithm 3.*

Proof. Both algorithms apply the same per-vertex threshold-support condition. The difference is only the schedule: SPIN recomputes the condition by full passes, while SPIN* uses Theorem 4.1 and Lemma 4.2 to update exactly the path contributions changed by the popped vertex. The path counters of SPIN* are support-equivalent to physically deleting finalised vertices from the CPI, so the bucket key of every unfinalised vertex is its residual threshold-support value clipped below by the current peel level. Initial values $h[u] = \deg^{(s)}(u)$ are upper bounds, and every update only decreases a key, so no vertex is ever assigned a value larger than its current residual threshold support above the current peel level. Consider the moment when the bucket cursor first reaches level k. The remaining vertices with key at least k form a super-level set in which each vertex has at least k qualifying s-clique witnesses, so each s-connected component of that set is a k-$(1, s)$-nucleus. Every vertex later popped while the cursor stays at level k therefore has core value at least k. When a vertex v is popped from bucket $B[k]$, it cannot belong to any $(k+1)$-$(1, s)$-nucleus: otherwise the exact residual threshold-support count at threshold $k+1$ would keep its key at least $k+1$ until it was removed. Thus the pop assigns $\kappa_s(v) = k$. By Lemma 5.3, κ_s is also the fixed point reached by SPIN. □ □

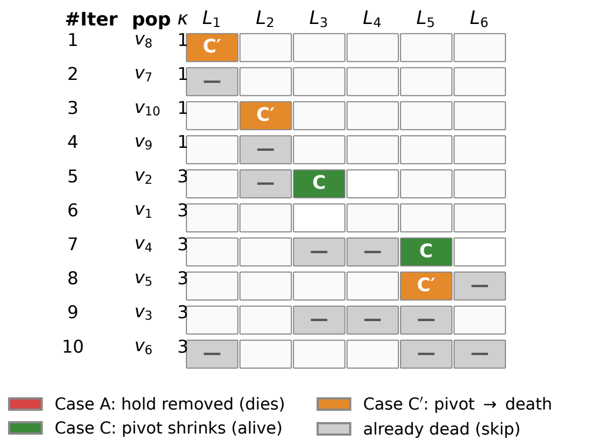

Figure 2: Incremental peel trace on the running example.

The runtime decomposes into the initialisation, the path-event walk on each peel, and the incremental support-refresh events the walk triggers:

Theorem 5.6 (SPIN* Complexity)**.** *Algorithm 4 runs in $O(\Sigma + U + n)$ time, where U is the total number of incremental support-refresh events over all affected path-mates.*

Proof. Initialization of vertex values and path counters costs $O(n + \Sigma)$. Each vertex–path incidence is visited when its vertex is finalized, so path-event processing costs $O(\Sigma)$. Each affected incremental refresh contributes one unit to U. Bucket moves and bucket pops are $O(1)$ amortized. The bucket cursor advances through at most the distinct h-values that are ever stored, which is bounded by n since at most one value is associated with each vertex at any time. The total time is therefore $O(\Sigma + U + n)$. □ □

The mutable baseline pays heap and residual-index maintenance costs on the same logical peel events. Figure 2 traces SPIN* on the running example: each row records one pop, and each cell marks the case (hold death, pivot shrink, or pivot-to-death) that the popped vertex triggers on the path indicated by the column header. The experiments in Section 8 measure the impact of replacing those operations with static path counters.

6 Parallel CPI Construction

Once SPIN* makes peeling counter-only, CPI construction becomes the load-bearing cost of the pipeline. The phase breakdown in Figure 5 shows that build time dominates the remaining runtime across the six-graph sweep, while peeling becomes a small slice of total time. This bottleneck shift is why construction is part of the algorithmic pipeline rather than a detached implementation detail. We parallelise the construction phase by exploiting two properties: the seed loop partitions emitted s-clique families along the degeneracy order, and the static output contract makes the merge a pointer-level concatenation.

Algorithm 5: ParaBuild — parallel CPI construction.

Input: Graph $G = (V, E)$; clique size s; thread count T.
Output: Static CPI $\mathcal{P}$.

1 compute degeneracy ordering σ on G and orient every edge from lower to higher rank;
2 partition the seed sequence $(v_1, \ldots, v_n)$ into T contiguous chunks $\sigma_1, \ldots, \sigma_T$;
3 **foreach** *thread* $t \in \{1, \ldots, T\}$ in parallel **do**
4 $\quad \mathcal{P}_t \leftarrow \emptyset$;
5 $\quad$ **foreach** *seed vertex* $v \in \sigma_t$ **do**
6 $\quad\quad \mathcal{P}_t \leftarrow \mathcal{P}_t \cup \textsc{ExpandPath}(N^+(v), \{v\}, \emptyset)$; `// Algorithm 1, seed v`
7 $\mathcal{P} \leftarrow \bigcup_{t=1}^{T} \mathcal{P}_t$;
8 **return** $\mathcal{P}$

Why the seed loop can be parallel. The CPI builder follows the standard degeneracy-ordered Bron–Kerbosch search with Tomita pivoting [9, 15, 44, 49]. Each s-clique has one owner: the smallest vertex of the clique in the degeneracy order. Therefore, the outer loop over seed vertices partitions the emitted s-clique families. No two seeds own the same s-clique.

Static CPI also changes what construction must produce. The mutable baseline builds residual tree state because later peeling may edit that state. Our $r = 1$ pipeline never edits the stored hold or pivot sets. The builder therefore only has to emit the path set $\mathcal{P}$; no per-path mutation contract has to be maintained across the parallel boundary.

Independent per-thread enumeration. ParaBuild takes G, s, and a thread count T. Each thread owns a contiguous range of seed vertices and runs the same seed-rooted enumeration as the sequential builder on each of its seeds, accumulating the emitted paths into its own output set $\mathcal{P}_t$. Because the degeneracy ordering gives every s-clique a unique smallest-rank owner, the sets $\{\mathcal{P}_t\}_t$ are pairwise disjoint, so the parallel output is the disjoint union $\mathcal{P} = \bigcup_t \mathcal{P}_t$ and the merge is purely the set union — no synchronisation across threads is needed during enumeration, and no path needs to be deduplicated or rewritten after enumeration. The expensive work is the Bron–Kerbosch search under each seed; the merge contributes no per-path arithmetic.

Algorithm 5 runs the same seed-rooted ExpandPath enumeration as Algorithm 1, but partitions the n seed vertices into T contiguous chunks (Line 2) and processes each chunk in its own thread (Lines 3–6). Because every s-clique has a unique smallest-rank owner under σ, the chunks induce a partition of $\mathcal{P}$: each path is emitted by exactly one thread, so the per-thread outputs $\mathcal{P}_t$ are pairwise disjoint and their union $\bigcup_t \mathcal{P}_t$ is the same set of paths Algorithm 1 would have produced sequentially (Line 7). The choice of T, the partition of seeds, and the in-memory layout used to store $\mathcal{P}$ are implementation knobs; they affect throughput but not the output set.

Correctness. ParaBuild is a parallel schedule of the same seed-owned enumeration. Seed ownership gives disjoint emitted s-clique families. The deterministic merge changes only the storage order, not the represented path families. The resulting path set satisfies Definition 2.5. Therefore, the support formula of Lemma 2.6 applies unchanged.

Scope. The design parallelizes construction because construction is the phase exposed by the static-CPI pipeline. It does not claim a lower bound showing that SPIN* cannot be parallelized. It also does not claim ideal scaling. The scaling evidence is reported in Figure 6.

7 Core-Value Hierarchy Construction

Because the static-CPI invariant of Section 4 keeps every leaf intact through peeling, the hierarchy is no longer a separate decomposition problem. After the core values are known, every CPI leaf encodes the same family of s-cliques both before and after the peel, so the elder-rule join tree of Definition 2.3 can be emitted by generating a short list of merge events per leaf that tracks the levels at which its encoded s-cliques become alive and scanning those events in descending level with a union–find, in $O(\Sigma \log \Sigma)$ time, without re-enumerating a single clique. Algorithms that mutate the CPI during peeling cannot reuse it for this purpose: they must either re-run s-clique enumeration at each level or maintain one parallel disjoint-set forest per core level during peeling, which blows up memory by an order of magnitude (Section 8, Figure 9). For any level k, the super-level set is $V_k = \{v \mid \kappa_s(v) \geq k\}$ and Definition 2.3 measures connectivity inside V_k through s-clique witnesses contained in V_k; BuildHier (Algorithm 6) materialises that filtration from the preserved leaves.

Per-clique events from each leaf. A CPI leaf P encodes the family $\{V_h(P) \cup X \mid X \subseteq V_p(P), |X| = \eta_P\}$ of s-cliques (Definition 2.5). A given encoded clique $V_h(P) \cup X$ is alive in V_k iff $\min_{v \in V_h(P) \cup X} \kappa_s(v) \geq k$, so different cliques of the same leaf may first appear at different levels. For each leaf we therefore generate a short ordered list of *merge events* that exactly track the level at which a new encoded clique enters V_k. Let $h_{\min} = \min_{v \in V_h(P)} \kappa_s(v)$ (or $+\infty$ if $V_h(P) = \emptyset$) and sort $V_p(P)$ as $p_1, \ldots, p_m$ with $\kappa_s(p_1) \geq \cdots \geq \kappa_s(p_m)$. When $\eta_P = 0$ the leaf encodes the single clique $V_h(P)$, fired at level $h_{\min}$ with member set $V_h(P)$. When $\eta_P \geq 1$, the first event fires at level $k^{(\eta_P)} = \min(h_{\min}, \kappa_s(p_{\eta_P}))$ and unifies $V_h(P) \cup \{p_1, \ldots, p_{\eta_P}\}$ — the highest-birth encoded clique. For each $i = \eta_P + 1, \ldots, m$ a follow-up event fires at level $k^{(i)} = \min(h_{\min}, \kappa_s(p_i))$ and unifies the new pivot p_i into the leaf's already-formed component (concretely with p_1 as anchor), because adding p_i activates the encoded cliques that contain p_i together with $\eta_P - 1$ already-active pivots. The list of events from a single leaf has length $|V_p(P)| - \eta_P + 1$ when $\eta_P \geq 1$, and length 1 when $\eta_P = 0$ (the leaf encodes the single clique $V_h(P)$ regardless of $V_p(P)$); in both cases the events fire at nonincreasing levels.

Reverse scan over events. BuildHier concatenates events across all leaves and processes them in nonincreasing level. Each event (k, S) unifies the vertices in S via union–find with the elder rule (the earliest-born component absorbs the others, and each younger component dies into the elder at level k). A vertex first appearing in some event with $k_{\text{event}} \geq 1$ is born at the highest level at which it is unified; vertices that never appear ($\kappa_s(v) = 0$) are omitted from the hierarchy.

Algorithm 6 runs in two phases. Phase 1 (Lines 1–10) generates merge events: for each leaf with at least one encoded s-clique, the algorithm sorts its pivot set by κ_s in descending order and emits one event for the highest-birth encoded clique (size $|V_h| + \eta_P$) followed by a series of unit events that each splice one lower-κ pivot into the leaf's already-formed component. Phase 2 (Lines 11–23) processes events in nonincreasing level: for each event (k, S), the union–find returns the root set R of S, emits a new hierarchy node for any root

Algorithm 6: BuildHier — elder-rule join tree from a static CPI.

Input: Static CPI $\mathcal{P}$ and core values $\kappa_s(v)$ for every $v \in V$, both produced by Algorithm 4 (SPIN*).
Output: Elder-rule join tree T of the s-clique-connected super-level filtration.

1 $\mathcal{E} \leftarrow [\,]$; `// merge events` (k, S)`: unify the vertex set` S `at level` k
2 **foreach** *leaf* $P \in \mathcal{P}$ *with* $|V_h(P)| + |V_p(P)| \geq s$ **do**
3 $\quad h_{\min} \leftarrow \min\{\kappa_s(v) \mid v \in V_h(P)\}$; `// +∞ if` $V_h(P) = \emptyset$
4 $\quad$ **if** $\eta_P = 0$ **then**
5 $\quad\quad$ append $(h_{\min}, V_h(P))$ to $\mathcal{E}$;
6 $\quad$ **else**
7 $\quad\quad$ sort $V_p(P)$ as $p_1, \ldots, p_m$ with $\kappa_s(p_1) \geq \cdots \geq \kappa_s(p_m)$;
8 $\quad\quad$ append $\left(\min(h_{\min}, \kappa_s(p_{\eta_P})),\, V_h(P) \cup \{p_1, \ldots, p_{\eta_P}\}\right)$ to $\mathcal{E}$; `// first event`
9 $\quad\quad$ **foreach** $i \leftarrow \eta_P{+}1$ **to** m **do**
10 $\quad\quad\quad$ append $\left(\min(h_{\min}, \kappa_s(p_i)),\, \{p_1, p_i\}\right)$ to $\mathcal{E}$; `//` p_i `joins via anchor` p_1
11 sort $\mathcal{E}$ by nonincreasing level (ties broken by event order);
12 initialise union-find U over V; birth$[r] \leftarrow +\infty$ for every root r;
13 $T \leftarrow$ empty forest;
14 **foreach** *event* (k, S) *in sorted order* **do**
15 $\quad R \leftarrow \{\text{Find}(x) \mid x \in S\}$;
16 $\quad$ **foreach** *root* $r \in R$ **do**
17 $\quad\quad$ **if** *birth*$[r] = +\infty$ **then**
18 $\quad\quad\quad$ emit a hierarchy node n_r with $k_{\text{birth}} \leftarrow k$; birth$[r] \leftarrow k$;
19 $\quad$ **if** $|R| \geq 2$ **then**
20 $\quad\quad$ sort R descending by $(\text{birth}[r], \text{id}(r))$; let $r^\star$ be the elder ; `// elder rule`
21 $\quad\quad$ **foreach** $r \in R \setminus \{r^\star\}$ **do**
22 $\quad\quad\quad$ attach n_r as child of $n_{r^\star}$ at level k in T ; `// younger dies`
23 $\quad\quad$ Union all of R into a single component;
24 **return** T

born here for the first time (Lines 16–18), and applies the elder rule to merge multiple roots into the eldest (Lines 19–23). The returned forest T is the elder-rule join tree of the s-clique-connected super-level filtration of (V, κ_s).

Correctness. Fix a level k and let C be any s-clique alive in V_k. Uniqueness of the CPI encoding (Definition 2.5) gives the leaf P such that $C = V_h(P) \cup X$ with $X \subseteq V_p(P)$, $|X| = \eta_P$. All members of X are in V_k, so each is among the top $|X| = \eta_P$ pivots of P once $V_p(P)$ is restricted to V_k; hence at the moment C enters the filtration (level $\min_{v \in C} \kappa_s(v)$) the corresponding event has either already fired (if C uses only the top η_P pivots) or fires at exactly that level (when C uses the new pivot p_i, the event $(k^{(i)}, \{p_1, p_i\})$ joins p_i to the leaf's component). Either way the vertices of C are in one component at level $\min_{v \in C} \kappa_s(v)$, so every Definition 2.3 merge is performed at the correct level. The elder rule preserves the component with earlier birth and attaches the younger one in the join tree [14].

Complexity. Phase 1 (event generation) scans each leaf once and sorts its pivot set: $O(\Sigma + \sum_P |V_p(P)| \log |V_p(P)|) \subseteq O(\Sigma \log L_{\max})$ ($L_{\max}$ = maximum path size, Section 5.1). The number of events $|\mathcal{E}|$ is at most $\sum_P \max(1, |V_p(P)| - \eta_P + 1) \leq \Sigma$, and sorting them by level costs $O(\Sigma \log \Sigma)$. Phase 2 (event processing) performs at most $\sum_P (|V_h(P)| + \eta_P) + \sum_{P:\eta_P \geq 1} 2(|V_p(P)| - \eta_P) \leq 2\Sigma$ union-find operations in total (the first event of each leaf unifies $|V_h| + \eta_P$ elements; for $\eta_P \geq 1$ leaves each of the $|V_p(P)| - \eta_P$ follow-up events unifies a pair), costing $O(\Sigma\, \alpha(n))$. Total time $O(\Sigma \log \Sigma)$, with no clique re-enumeration: BuildHier touches each CPI incidence a constant number of times.

Memory. Beyond the union-find of size $n + |\mathcal{P}|$ (the implementation uses each CPI leaf as an auxiliary connector node so that one union per event suffices to bind every member of S), the routine needs a list of vertices per leaf to fire unions. SPIN*'s vertex-to-path inverse CSR (Section 5) supports a single bucket-CSR pass that materialises the per-leaf lists in $O(\Sigma)$ temporary memory; this scratch is freed on return. The static CPI budget of Section 8 therefore holds throughout peeling proper and is only briefly elevated during the hierarchy emit.

Consequence. The clique-coherent core-value hierarchy is not a separate decomposition problem in this pipeline. It is a byproduct of the static CPI surviving peeling: each preserved leaf contributes a short sequence of merge events tracking when its encoded cliques become alive, and a single descending union–find pass over those events produces the join tree. Methods that mutate the CPI during peeling cannot reuse it for hierarchy emit and need a second clique-enumeration pass [39].

Section 8 (Exp-7) compares BuildHier against the only feasible baseline at r=1 — CND's general (r, s) level-DSU hierarchy specialised to r=1 — and confirms that BuildHier uses up to 43× less memory (gmean 13× over 24 matched cells) and up to 35× less hierarchy-emission wall time, with the gap growing at larger s where the witness mass dominates.

8 Experiments

Exactness comes from the invariants and fixed-point equivalence proved earlier; the experiments measure cost. **Exp-1** measures end-to-end time and peak memory for SPIN*, the practical static-CPI pipeline. **Exp-2** separates construction from peeling, because counter-only peeling shifts rather than removes all cost. **Exp-3** tests how the static construction contract scales with thread count. **Exp-4** measures SPIN as the direct fixed-point baseline. **Exp-5** pushes the pipeline to a billion-edge graph (`com-friendster`, 1.806B edges). **Exp-6** sweeps a vertex-induced subsampling ratio to test how runtime and memory scale with input size. **Exp-7** compares hierarchy emission against CND's level-DSU baseline.

Hardware and software. All experiments run on an Intel Xeon Gold 6248*R* server with 503 GB DDR4 memory. The operating system is Ubuntu 22.04. The code[1] is compiled with Clang 18.1 using `-O3 -march=native`. It is linked against TBB 2021.10, Boost 1.83, and `robin_hood::unordered_flat_map`. Single-threaded measurements use one core. Parallel-construction measurements use the thread counts reported in Figure 6. Time and memory measurements are averaged or summarized over repeated runs as stated in each experiment.

Datasets. Table 2 lists the benchmark graphs. The set covers collaboration, e-commerce, social, and web graphs. The largest graph is `com-friendster` with 1.8B edges. The benchmark sweep on the ten smaller graphs yields 307 matched cells where both SPIN* and CND complete successfully, and the headline speedup numbers are reported over the matched subset; `com-friendster` is run standalone (Section 8) because CND cannot complete on it.

Algorithms and methodology. CND [49] is the mutable-CPI implementation described in Section 3, restricted to $r = 1$ for this comparison; SPIN* is the static-CPI pipeline of this paper; SPIN

[1]We will release the code, bench harness, and result CSVs under an anonymised public archive at submission and de-anonymise the repository on acceptance.

Table 2: Datasets.

Graph	$\|V\|$	$\|E\|$	avg. deg.
`com-amazon` [48]	334,863	925,872	5.5
`com-dblp` [48]	317,080	1,049,866	6.6
`twitter` [48]	81,306	1,342,296	33.0
`web-Stanford` [48]	281,903	1,992,636	14.1
`com-youtube` [48]	1,134,890	2,987,624	5.3
`web-Google` [48]	875,713	4,322,051	9.9
`wiki-Talk` [48]	2,394,385	4,659,565	3.9
`web-it-2004` [48]	509,338	7,178,413	28.2
`soc-pokec` [48]	1,632,803	22,301,964	27.3
`com-orkut` [48]	3,072,441	117,185,083	76.3
`com-friendster` [48]	65,608,366	1,806,067,135	55.1

Table 3: Per-incidence memory budget at $\Sigma \gg n$, where $\Sigma = \sum_{P\in\mathcal{P}} \big(|V_h(P)| + |V_p(P)|\big)$ is the total vertex–path incidence count (Section 2) and B is the number of distinct h-values that ever appear in a bucket (sparse buckets, $B \le n$).

Component	**Baseline**	SPIN*
BK recursion tree T	8Σ	0
Path-to-vertex storage (V_h, V_p)	0	4Σ
Per-path vertex set H_P	32Σ	—
Reverse vertex-to-path map	48Σ	—
Vertex-to-path incidence	—	4Σ
Support array	$4n$	$4n$
Heap / bucket array	$24n$	$16n + 8B$
Analytical Σ-coefficient	$\approx 88\Sigma$	$\approx 8\Sigma$
Measured: `com-youtube`, s=5	642 MB	387 MB
Measured: `web-it-2004`, s=3	556 MB	270 MB
Measured: `web-Stanford`, s=10	261 MB	146 MB

is the direct fixed-point reference of Section 5.2. All three solvers consume the same input edge list and run on the same hardware; the comparison therefore isolates the cost of CPI mutability against static-state peeling. For each matched $(graph, s)$ pair, we record the end-to-end time (load, degeneracy order, CPI construction, peel, output) and the program's memory usage.

Exp-1: peel time and memory. Across the ten graphs (Figures 3–4), SPIN* achieves a peel-time speedup over CND of gmean 13.50× / max 44.76×, and a memory-usage ratio (CND/SPIN*) of gmean 1.55× / max 3.92×; SPIN* sits below CND on every cell of the peel-time plot. SPIN (Section 5.2) is included as the reference fixed-point solver; SPIN* beats it as well on all but the easiest cells, with the full comparison in Exp-4 below. The end-to-end (wall-clock) gap is smaller (gmean 1.10×) because the shared CPI construction phase dominates total runtime once peeling is cheap; this is the bottleneck shift that motivates ParaBuild in Section 6.

The memory improvement is explained by the incidence-level budget in Table 3. The mutable baseline stores path hash sets and reverse hash maps. SPIN* stores flat path-to-vertex and vertex-to-path arrays plus path counters. The analytical per-incidence gap is larger than the observed memory gap because graph adjacency, allocator overhead, and per-vertex structures do not scale with Σ.

Exp-2: phase breakdown. Counter-only peeling is cheap; we measure how much of the remaining cost lies in CPI construction. The phase study sweeps $s \in \{3, 5, 7, 9, 11, 13, 15\}$ on six graphs (Figure 5, smallest to largest by edge count). The orange peel slice on top of each bar shrinks with s on every graph past a small-s transient (a few cells have a non-monotone bump at low s where the peel cost is already dominated by setup): on the medium and large graphs peel falls to a few percent of total time by s=9 and below 1% by s=11, and stays below 0.1% on `web-it-2004` throughout the sweep. The bottleneck has shifted onto construction, which is why ParaBuild is part of the algorithmic pipeline rather than a detached engineering detail.

Exp-3: parallel scaling of CPI construction. ParaBuild exposes seed-level parallelism because each seed can stream its emitted paths into thread-local storage. Figure 6 reports build time on six graphs as the thread count grows from T=1 to T=64 (median of three runs per cell), with one panel per graph showing the representative s=3 curve; each curve ends at its optimum thread count, and graphs with T=1 baseline below 500 ms are omitted as uninformative. On the densest large graphs the optimum lies at T=24–48; on smaller graphs and on sparser large graphs whose per-thread work is small (e.g. `wiki-Talk`) it sits around T=16. The headline single-cell speedup is 23.4× at T=24 on `web-it-2004` s=3 (92.3 s $\to$ 3.95 s).

Exp-4: SPIN* vs. SPIN on the same fixed point. SPIN is the literal single-thread solver for the same equations that SPIN* realizes by deletion events. The two algorithms compute the same core values (Lemma 5.5), so the comparison isolates the effect of replacing repeated full-graph passes with incremental updates. Both solvers read paths from the same static index; the SPIN reference therefore reflects the algorithmic cost of repeated full-pass fixed-point iteration, not a difference in how the index is stored. Across the 272 matched $(graph, s)$ cells, SPIN* delivers a 30.4× geometric-mean peel-time speedup over SPIN, with maximum 2,369× on `web-it-2004` at $s = 53$, where SPIN repeatedly sweeps every vertex while SPIN* finishes after a few path-event updates. On the easiest cells SPIN* can be slightly slower (minimum ratio 0.62×) because the bucket-queue setup and live-path bookkeeping add fixed overhead that SPIN's tight inner loop avoids when one or two passes already converge. On the largest inputs SPIN hits the 1h cap (`com-orkut` for $s \ge 3$, `wiki-Talk` for the single cell $s = 10$); SPIN* finishes the same configurations in seconds to minutes.

SPIN's memory profile is essentially identical to SPIN*'s on every paired cell, because both share the same static index: the geometric-mean memory ratio (SPIN/SPIN*) is 0.97 over the 291 memory-paired cells (more than the 272 time-paired cells because some non-finishing SPIN runs still recorded a peak-RSS value), and SPIN is slightly smaller than SPIN* on 83.8% of them. The reason is structural: SPIN*'s extra state beyond the shared CPI consists of a per-path counter cache (~ 13 bytes per leaf) and a peel-order bucket queue (~ 17 bytes per vertex), which SPIN does not maintain because it scans every vertex each round. The sub-percent memory advantage of SPIN is therefore the price SPIN* pays to avoid repeated full-graph passes; the trade is the textbook "space for time" choice, with the time term dominating by orders of magnitude.

The paper carries both algorithms because they serve different roles: SPIN is the direct semantic formulation, SPIN* is its event-driven realisation.

Exp-5: scaling to a billion-edge graph. The ten-graph benchmark stops at `com-orkut` (117M edges); to push the pipeline an order of

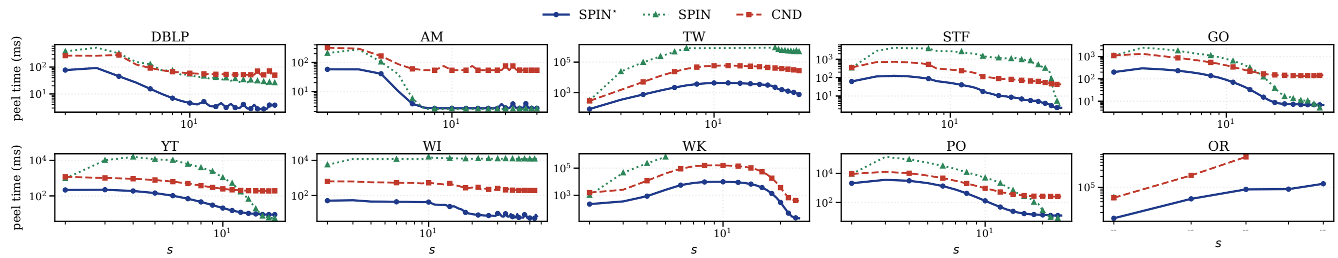

Figure 3: Peel time across ten graphs (three algorithms: SPIN★, SPIN, CND**).**

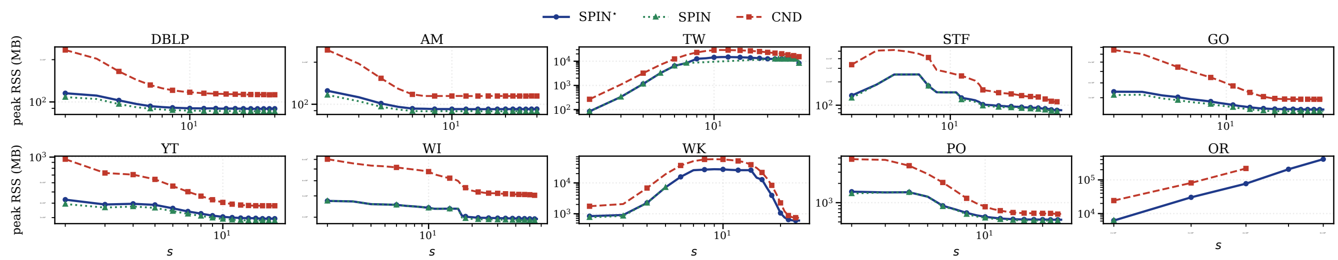

Figure 4: Peak memory across ten graphs (three algorithms: SPIN★, SPIN, CND**).**

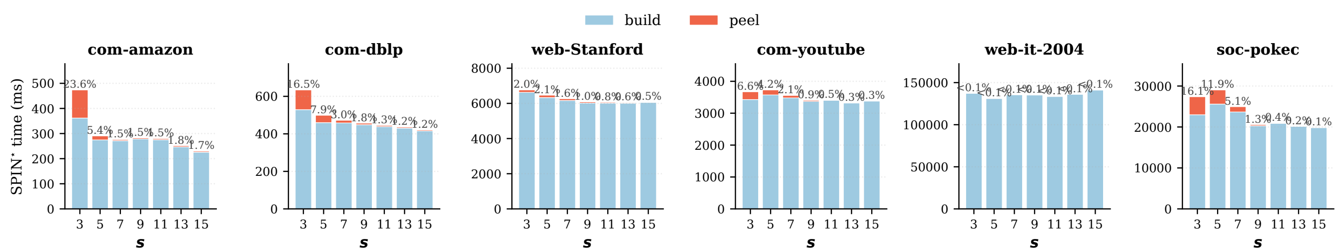

Figure 5: SPIN★ **phase breakdown on six graphs: build (blue, bottom) vs peel (orange, top). Linear scale; the peel share is printed above each bar.**

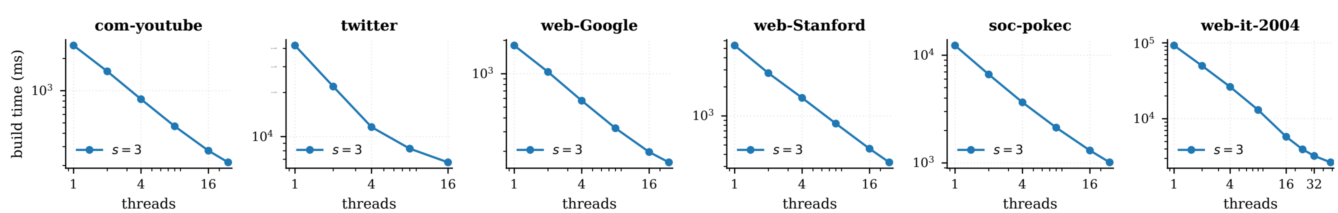

Figure 6: ParaBuild **build time vs. thread count; one curve per representative** s**, ending at its optimum** T**.**

magnitude further, we run SPIN★ with T=24 on `com-friendster` (65M vertices, 1.806B edges, the largest SNAP graph available). The headline result is coverage: SPIN★ completes the $(1, s)$-nucleus decomposition at every $s \in \{2, 3, 4, 5, 6, 11\}$ we tested, whereas CND succeeds only at s=2 and is OOM-killed (exceeds the 503 GB budget during CPI construction) at every $s \geq 3$ we tested (Figure 7). On the one matched cell (s=2), SPIN★ peels in 344 s against CND's 1,775 s (5.2× faster) and uses 160 GB against 400 GB (2.5× less); the memory headroom is what lets SPIN★ continue past the budget where CND cannot. At higher s the static index stays tractable: at s=6 it stores $\Sigma \approx 2.26 \times 10^{10}$ vertex–path incidences and peels the resulting structure in 651 s; at s=11, $\Sigma \approx 1.68 \times 10^{10}$ and the peel finishes in 167 s. For practical $(1, s)$ analysis at $s \geq 3$ — the regime where higher-order cohesion measures separate from the k-core — SPIN★ is the only one of the two solvers that runs at all on this graph. End-to-end runtime is build-dominated (5–7 thousand seconds of construction vs. 167–676 seconds of peel), confirming that the bottleneck shift carries over to the billion-edge regime and that the parallel builder of Section 6 is what makes the pipeline usable at this scale.

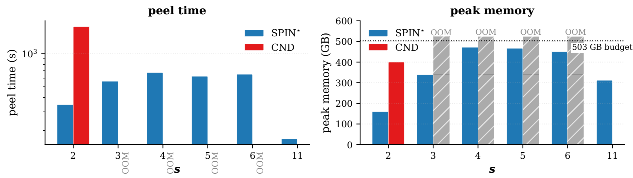

Figure 7: `com-friendster` (1.806B edges, the largest SNAP graph): SPIN* completes the $(1, s)$ decomposition at every tested $s \in \{2, 3, 4, 5, 6, 11\}$ while CND only completes at s=2 and exceeds the 503 GB budget at every $s \geq 3$ tested. On the one matched cell, SPIN* peels in 344 s vs. 1,775 s (5.2×) using 160 GB vs. 400 GB (2.5× less).

Exp-6: scaling with input size. The benchmark cells fix a graph and sweep s; they do not test how runtime and memory respond to growing the input itself. To probe scaling, we shrink each graph by vertex-induced subsampling: for each ratio $\rho \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$ we keep $\lceil \rho\, n \rceil$ vertices selected uniformly at random with a fixed seed and take the induced subgraph (only edges with both endpoints kept). The same seed makes the subsamples nested (20% ⊂ 40% ⊂ ⋯ ⊂ 100%), so each larger input contains the previous one and the scaling curves compare increasingly large induced graphs. We pick vertex-induced rather than random-edge subsampling because SPIN*'s work scales with the encoded s-clique count Σ; removing edges uniformly drops Σ super-linearly because each missing edge can destroy many cliques, producing jagged curves, while a vertex-induced subsample preserves clique structure inside the kept vertex set. We re-run SPIN* on six graphs spanning over two orders of magnitude in edge count (`com-dblp`, `com-youtube`, `web-Stanford`, `web-Google`, `soc-pokec`, `com-orkut`) for $s \in \{2, 3, 5, 7, 9, 11, 13, 15\}$ on each subsample. Figure 8 reports end-to-end time (top row) and memory usage (bottom row) as the ratio grows. On every graph SPIN*'s curves are ordered cleanly by ratio in the visible range of both metrics: denser subsamples take longer and use more memory, with no meaningful inversions on the log-scale plots, indicating that the static-CPI pipeline scales smoothly with input size from sparse collaboration graphs to dense social graphs. The densest graph, `com-orkut`, also exposes the natural ceiling of vertex-centric $(1, s)$-nucleus decomposition at high s: the 100% curve cuts off near s=5 because the encoded clique count exceeds what a 30-minute budget can peel.

Exp-7: hierarchy emission against the level-DSU baseline. BuildHier (Section 7) consumes the preserved CPI once peeling finishes; the only available $(1, s)$ baseline that produces the same hierarchy is CND's general (r, s) level-DSU hierarchy specialised to r=1, which carries one parallel disjoint-set forest per distinct attained core value ($K = |\{\kappa_s(v) : v \in V\}|$, typically $\leq n$ and orders of magnitude smaller than the per-vertex clique count $\kappa_{\max}$; denoted K in the $O(K \cdot \Sigma)$ bound below) during the peel. We compare four algorithm variants on four representative graphs (Figure 9), sweeping $s \in \{3, 5, 7, 9, 11, 13, 15\}$: SPIN* (no hierarchy), SPIN*+BuildHier (our $O(\Sigma \log \Sigma)$ post-peel emit), CND (no hierarchy), and CND+**Hier** (level-DSU hierarchy during peel). All four solve the same problem; the comparison isolates the cost of hierarchy emission and of the underlying peel strategy. Three observations follow: (i) BuildHier adds modest memory on top of SPIN*'s peel state (gmean 1.26×, max 2.41× on the densest small-s cells; blue and green curves nearly overlap on every panel), while CND+Hier uses up to 43× more memory than SPIN*+BuildHier on matched cells (gmean 13× over 24 cells; orange far above green); (ii) even *without* emitting a hierarchy, CND (red) already uses more memory than SPIN*+BuildHier (green), and on `wiki-Talk` at $s \geq 9$ both CND variants exceed the 503 GB budget before producing a result, while ours finishes the entire sweep; (iii) the hierarchy emit is also cheaper at the larger-s cells where the witness mass dominates: BuildHier is up to 35× faster than CND+Hier's union work (gmean 7.7× over the 24 cells); on the smallest-s cells where the hierarchy emit is already sub-second the two are comparable, consistent with the $O(\Sigma \log \Sigma)$ vs. $O(K \cdot \Sigma)$ bounds.

Limitations. The static-CPI invariant is specific to r=1. For general (r, s) with $r \geq 2$, edge or higher-order peel events can destroy the cliques that a path stores, and the present pipeline does not subsume the mutable-CPI machinery of [49]. Memory is the binding resource on the densest graphs: Σ grows superpolynomially with s on dense inputs, so on a fixed-memory machine the largest reachable s is determined by the available RAM rather than by an algorithmic limit. Support counters use $\binom{p}{\eta}$ which exceeds 2^{63} at moderate p; the implementation guards the threshold pass with 128-bit accumulators when the count overflows 64 bits, but applications that need exact support magnitudes (rather than ranks) must use the same guard. Finally, the experiments here run single-machine; distributed-memory deployments and dynamic-graph updates are not evaluated.

Reproducibility. All experiments are scripted: build commands, dataset URLs, and bench drivers are version-controlled alongside the source. Each binary run is wrapped with `/usr/bin/time -v` so memory usage, page faults, and exit status are captured in every CSV row.

Takeaway. SPIN* is faster and lighter than CND on the 307 matched cells, the construction phase emerges as the new dominant cost once peeling is counter-only, ParaBuild addresses that shift, and SPIN confirms the direct fixed-point view's role as semantic reference rather than practical solver.

9 Case Studies

The experiments quantify cost; the case studies establish that the r = 1 specialisation is worth optimising in the first place. On the ground-truth tasks reported here, $(1, s)$ matches the quality of higher-r nuclei on the evaluated metrics at much lower cost, a claim scoped to the datasets and metrics below.

Setup. For cross-r comparisons we use the standard (r, s)-nucleus parameterisation [39]; the paper's algorithmic results apply only to r = 1, while the r = 2 and r = 3 scores serve as baselines from the same family. Because edge- and triangle-level scores need to be compared with vertex rankings, we project them by assigning each vertex the maximum score of an incident r-clique, then rank descending with ties broken uniformly.

The ground-truth datasets are `com-dblp` and `com-youtube` from SNAP [48]. `com-dblp` has 317,080 vertices, 1,049,866 edges, and 5,000 venue-based ground-truth communities. `com-youtube` has 1,134,890 vertices, 2,987,624 edges, and 5,000 ground-truth communities.

We use three ranking metrics. Precision@K is the fraction of top-K vertices that belong to at least one ground-truth community. rec50

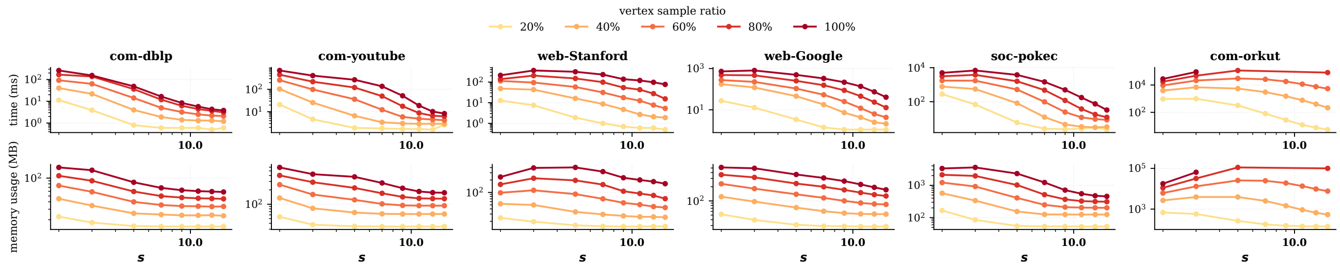

Figure 8: Vertex-induced scalability: time (top) and memory usage (bottom).

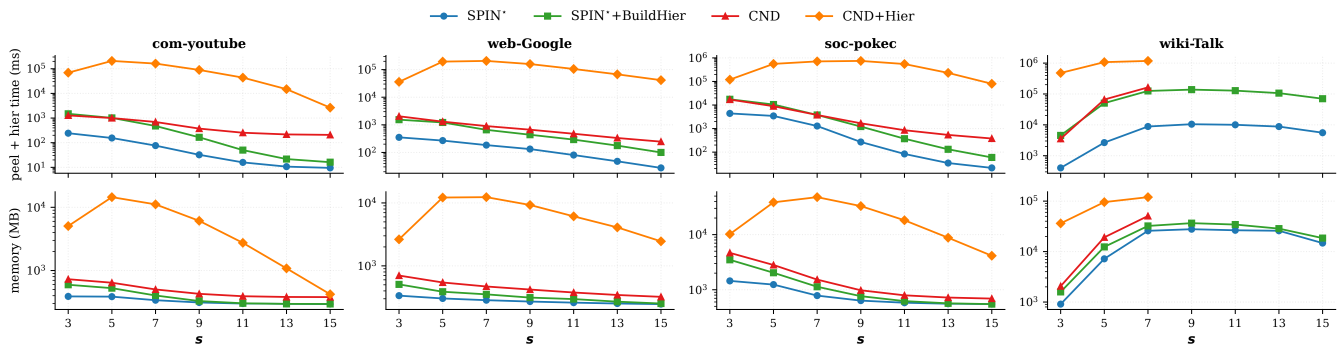

Figure 9: Hierarchy bench on four graphs: wall time (top) and peak memory (bottom).

Table 4: Active set on `com-dblp` as s grows.

s	nonzero $\|V\|$	% of n	max core	top-100 min	top-1k min
2	317,080	100.00	1.13×10^{2}	1.13×10^{2}	3.10×10^{1}
3	269,181	84.89	6.33×10^{3}	6.33×10^{3}	4.65×10^{2}
4	194,957	61.49	2.34×10^{5}	2.34×10^{5}	4.50×10^{3}
5	127,805	40.31	6.44×10^{6}	6.44×10^{6}	3.15×10^{4}
8	36,412	11.48	3.86×10^{10}	3.86×10^{10}	2.63×10^{6}
10	19,354	6.10	5.97×10^{12}	5.97×10^{12}	2.02×10^{7}
15	6,767	2.13	2.74×10^{17}	2.74×10^{17}	2.65×10^{8}
20	3,396	1.07	1.69×10^{21}	1.69×10^{21}	1.41×10^{8}
25	2,069	0.65	2.18×10^{24}	2.18×10^{24}	2.63×10^{6}
30	1,358	0.43	7.61×10^{26}	7.61×10^{26}	4.65×10^{2}

is the number of communities with at least 50% of their vertices inside the top-K. Triangle density is the number of triangles per vertex in the induced top-K subgraph.

Case study 1: s is a granularity knob. The first question is what s changes when $r = 1$. On `com-dblp`, increasing s sharply shrinks the active set while preserving the very top of the ranking. Table 4 reports the active-set size and top-prefix thresholds.

The active set shrinks from all vertices at $s = 2$ to 1,358 vertices at $s = 30$. This is a 234× reduction. At the same time, the maximum core value spans many orders of magnitude because high-s supports are binomial counts. The practical interpretation is simple. s controls how aggressively the ranking filters the graph.

Case study 2: $(1, s)$ improves the k-core ranking. On `com-dblp`, the first non-trivial clique support already improves the ground-truth ranking over k-core. Table 5 reports top-10,000 quality.

$(1, 3)$-core raises precision from 0.504 to 0.523 and rec50 from 94 to 114. For $s \in \{3, 5, 10\}$, the top-10,000 quality is identical in

Table 5: Ground-truth quality on `com-dblp`.

Method	top-K	Precision	rec50	tri/v
k-core	10,000	0.504	94	112.3
$(1, 3)$-core	10,000	0.523	114	113.6
$(1, 5)$-core	10,000	0.523	114	113.5
$(1, 10)$-core	10,000	0.523	114	113.4
$(1, 15)$-core	6,767	**0.548**	65	**154**
$(1, 30)$-core	1,358	0.507	4	**523**

this table. For larger s, the active set drops below 10,000, so the comparison changes from ranking quality at fixed K to density of the surviving subgraph. $(1, 30)$-core reaches 523 triangles per vertex on 1,358 vertices.

Case study 3: higher r does not buy quality on the studied graphs. Edge- and triangle-level nuclei sit between $(1, s)$ and explicit clique enumeration on the cost axis; whether they buy quality at that cost is an open question on the same datasets. Fixing s and comparing $r \in \{1, 2, 3\}$ across `com-dblp` (s=10) and `com-youtube` ($s \in \{5, 10, 15\}$), Figure 10 shows the answer at a glance: within each cell the three quality bars are nearly equal heights (left panel), while the time bars span 1–3 orders of magnitude (right panel, log scale). On `com-dblp` at s=10, $(1, 10)$ and $(3, 10)$ agree on rec50 (114 vs. 114) while r=1 is 178× faster (13.4 ms vs. 2380 ms); on `com-youtube` the same pattern repeats for every s, with r=1 within 5 rec50 points of r=3 (at s=5: 283 vs. 288; at s=10 and s=15: identical) and between 9.8× and 68× faster than r=3 (and 6.2–10.3× faster than r=2). r=1

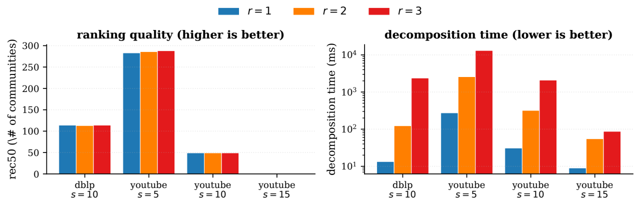

Figure 10: Cross-r comparison on `com-dblp` (s=10) and `com-youtube` ($s \in \{5, 10, 15\}$). Left: ranking quality (rec50, higher is better) is nearly identical across r. Right: decomposition time (log scale, lower is better) is 1–3 orders of magnitude smaller at r=1.

Table 6: Nucleus hierarchy summary.

	$s = 3$			$s = 5$			$s = 10$			$s = 15$		
graph	br.	int.	d	br.	int.	d	br.	int.	d	br.	int.	d
com-dblp	10,900	426	4	6,050	278	4	751	20	4	197	8	4
com-youtube	13,507	353	4	714	4	3	12	1	2	1	0	1
web-Stanford	3,455	181	5	1,093	20	3	228	7	3	90	2	2

therefore gives the best observed quality–runtime tradeoff for vertex ranking on these graphs; this does not prove higher r is never useful, only that increasing r here mostly adds cost.[2]

Case study 4: s distils a neighbourhood into near-clique modules. The previous studies use global rankings. The ego-network visualisation tests whether the same s knob also controls local interpretability. Figure 11 draws the two-hop ego-network of a `com-dblp` anchor on a shared spring layout under four successive local s-clique witness views ($s = 3, 5, 7, 10$). A vertex of the ego is shown alive at s iff it participates in at least one s-clique entirely within the ego subgraph. This local view visualizes the clique witness condition that drives $(1, s)$-nucleus decomposition; it is not used as a separate correctness claim. The four largest maximal cliques fully contained in the ego ($K_{31}, K_{25}, K_{19}, K_{18}$) are highlighted in colour with callouts.

These four cliques are alive at every s in $\{3, 5, 7, 10\}$ (their sizes are all at least 10), so they form a backbone that persists across all four panels. What changes from panel to panel is the peripheral alive set: vertices alive only because of smaller cliques peel off as s grows. Alive vertex count shrinks monotonically (1,356 $\rightarrow$ 721 $\rightarrow$ 370 $\rightarrow$ 165 out of 1,452): from "most of the neighbourhood is in some triangle" at s=3, to "only the four backbone cliques and their immediate clique-shared periphery survive" at s=10. The ego thus distils from a coauthor neighbourhood into a few large, tightly-coordinated coauthor communities — the same cliques are visible throughout, but the larger s strips away the loose periphery and isolates them.

Case study 5: hierarchy profiles differ by domain. BuildHier turns the core values into a $(1, s)$-clique-connected nucleus hierarchy (Definition 2.3). This makes it possible to compare how quickly the nucleus hierarchy collapses across domains. Table 6 reports the three hierarchy statistics used here. Figure 12 visualizes the resulting join trees side by side.

`com-youtube` collapses quickly: from 13,507 branches at $s = 3$ to a single branch at $s = 15$. `com-dblp` keeps the largest branch counts and a constant maximum depth across s, reflecting strong nested clique structure. `web-Stanford` retains nontrivial branches at every clique size, including $s = 15$. Figure 12 makes the contrast visual at $s = 5$ by plotting branch persistence ($k_{\text{birth}} - k_{\text{death}}$) against the rank of each branch when sorted by persistence (log–log). `com-dblp`'s curve is a long flat tail: hundreds of branches retain persistence above 10^2 and thousands above 10^1, the signature of a deep nested hierarchy. `com-youtube`'s curve drops sharply after the first few ranks and plateaus near persistence 1, showing that only a handful of branches carry real depth. `web-Stanford` lies between the two. The hierarchy is therefore not only an output format. It gives a domain-level fingerprint of clique-witness nesting.

Takeaway. On the DBLP and YouTube ground-truth tasks, $r = 1$ matches higher-r quality metrics at much lower cost; the hierarchy and ego-network studies show the same decomposition also yields interpretable local structure and a per-domain fingerprint. The evidence is scoped to these datasets and metrics rather than a universal claim, but it is enough to justify optimising the exact $r = 1$ case.

10 Related Work

This paper uses prior ideas on core peeling, compact clique representations, local fixed-point characterisations, parallel construction, and hierarchy output; the new ingredient is the static-CPI invariant for $r = 1$.

Enumeration-based core and nucleus decomposition. The k-core was introduced by Seidman [41] and is supported by a long line of efficient algorithms [5, 6, 8, 25, 35, 36, 45–47, 50]. Higher-order extensions strengthen the edge witness: k-truss [1, 10, 12, 22] uses triangle support, and *nucleus decomposition* [39, 40] generalises the witness to arbitrary s-cliques by peeling r-cliques according to their support in s-cliques. Hypergraph generalisations adopt the same peeling paradigm under different cohesion definitions [4, 7, 26, 29]. We specialise to the vertex-centric $r = 1$ case and exploit a state invariant that does not hold for general r.

Compact clique representations. Bron–Kerbosch search [9], Tomita pivoting [44], and degeneracy ordering [15] are standard tools for clique enumeration; Pivoter's succinct clique tree compactly encodes clique counts [23], and the Clique Path Index extends this line to support general nucleus decomposition with mutating hold/pivot updates during peeling [49]. The static-CPI counter form of this paper shows that for vertex peeling these path edits never need to be materialised.

Local and parallel peeling. h-index characterisations of core values [21] were studied for local core computation [31, 38] and extended to distributed and parallel settings [35, 50]. In this paper, the fixed-point view is adapted to the implicit s-clique witnesses encoded by the CPI, and SPIN* provides its peel-order realisation. The difference from prior local h-index work is not the operator but the representation: the witness multiset is held in static CPI paths and updated by binomial support-loss formulas.

Parallel construction and clique counting. Parallel clique search typically distributes the recursive enumeration work; ParaBuild follows the same seed-level decomposition. Because the downstream peel never mutates path sets, construction partitions into thread-independent subproblems whose results combine by a deterministic merge — an engineering consequence of the $r = 1$ state model rather than a new clique-enumeration paradigm.

[2] At `com-youtube` s=15 the active set has 95 vertices, below the smallest SNAP community size, so rec50 is zero for every method.

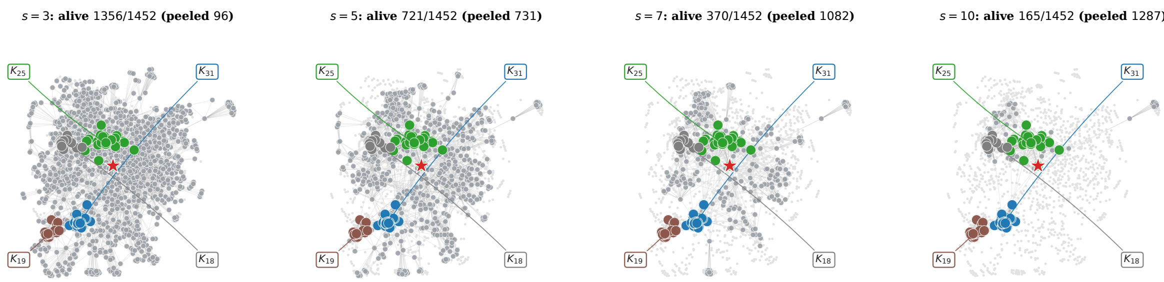

Figure 11: Same ego-network under four local s-clique witness views ($s = 3, 5, 7, 10$) on a shared spring layout. The four largest maximal cliques fully contained in the ego ($K_{31}, K_{25}, K_{19}, K_{18}$) form a backbone visible in every panel. As s grows the surrounding alive periphery shrinks from $1356 \to 721 \to 370 \to 165$ out of 1452, while the backbone cliques themselves remain intact.

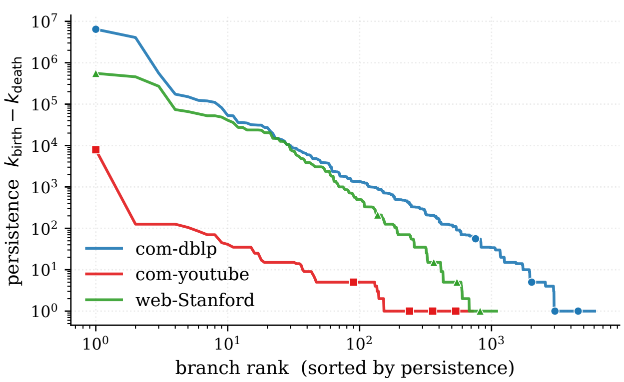

Figure 12: Branch persistence vs. rank at s=5 (log–log): long tail = deep hierarchy.

Hierarchical cohesive subgraphs. Nucleus decompositions naturally form hierarchies across core levels [37, 39, 40, 42, 43], and elder-rule join-tree extraction is standard in computational topology [14]. Prior nucleus pipelines extract such hierarchies by sorting the decomposition output or revisiting clique witnesses; BuildHier instead reuses the preserved static CPI, treating each leaf as a hyperedge born at its minimum core value, so the hierarchy is a byproduct of preserving the CPI through peeling.

Applications of k-core and nucleus decomposition. Core and nucleus decompositions are workhorses for community detection [13, 17, 18, 20], densest-subgraph discovery [19], influential-vertex and role analysis [11, 24, 30], network visualization [51], and downstream uses in social analysis [3, 16], biological networks [28, 33], complex-network analysis [27], anomaly and security applications [2, 34], and recommendation [32]. This paper inherits the same downstream uses; the contribution is making the vertex-centric $(1, s)$ output cheaper to obtain at higher s.

Empirical graph mining methodology. The evaluation follows common practice in cohesive-subgraph experiments: SNAP community ground truth, large public graphs, and runtime and memory comparisons [40, 48, 49].

11 Conclusion

For $(1, s)$-nucleus decomposition, vertex peeling preserves every edge among surviving vertices, so the path structure of the CPI stays valid throughout the peel and exact decomposition reduces to counter maintenance over a static symbolic index. SPIN* realises this fixed point in deletion order, ParaBuild parallelises the construction that is now the bottleneck, and BuildHier recovers the nucleus hierarchy from the static index. The matched configurations agree with the mutable baseline while reducing runtime and memory on the evaluated benchmarks, and the case studies show that vertex-centric $(1, s)$ matches higher-r quality metrics on community tasks at a fraction of the cost. The static-CPI invariant is specific to r=1, and extending it to restricted $r \geq 2$ settings, to streaming and dynamic graphs, and to GPU/distributed execution are the natural next steps.

References

[1] Esra Akbas and Peixiang Zhao. 2017. Truss-based community search: a truss-equivalence based indexing approach. *Proc. VLDB Endow.* 10, 11 (Aug. 2017), 1298–1309. https://doi.org/10.14778/3137628.3137640

[2] Leman Akoglu, Hanghang Tong, and Danai Koutra. 2015. Graph-based anomaly detection and description: a survey. *Data Mining and Knowledge Discovery* 29, 3 (2015), 626–688.

[3] J. Ignacio Alvarez-Hamelin, Luca Dall'Asta, Alain Barrat, and Alessandro Vespignani. 2005. Large scale networks fingerprinting and visualization using the k-core decomposition. In *Proceedings of the 18th International Conference on Neural Information Processing Systems* (Vancouver, British Columbia, Canada) *(NIPS'05)*. MIT Press, Cambridge, MA, USA, 41–50.

[4] Naheed Anjum Arafat, Arijit Khan, Arpit Kumar Rai, and Bishwamittra Ghosh. 2023. Neighborhood-Based Hypergraph Core Decomposition. *Proc. VLDB Endow.* 16, 9 (2023), 2061–2074. https://doi.org/10.14778/3598581.3598582

[5] Vladimir Batagelj and Matjaž Zaveršnik. 2003. An $O(m)$ Algorithm for Cores Decomposition of Networks. *arXiv preprint cs/0310049* (2003).

[6] Vladimir Batagelj and Matjaž Zaveršnik. 2011. Fast algorithms for determining (generalized) core groups in social networks. *Adv. Data Anal. Classif.* 5, 2 (July 2011), 129–145. https://doi.org/10.1007/s11634-010-0079-y

[7] Ginestra Bianconi and Sergey N Dorogovtsev. 2024. Nature of hypergraph k-core percolation problems. *Physical Review E* 109, 1 (2024), 014307.

[8] Francesco Bonchi, Francesco Gullo, Andreas Kaltenbrunner, and Yana Volkovich. 2014. Core decomposition of uncertain graphs. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (New York, New York, USA) *(KDD '14)*. Association for Computing Machinery, New York, NY, USA, 1316–1325. https://doi.org/10.1145/2623330.2623655

[9] Coen Bron and Joep Kerbosch. 1973. Algorithm 457: finding all cliques of an undirected graph. *Commun. ACM* 16, 9 (Sept. 1973), 575–577. https://doi.org/10.1145/362342.362367

[10] Yulin Che, Zhuohang Lai, Shixuan Sun, Yue Wang, and Qiong Luo. 2020. Accelerating truss decomposition on heterogeneous processors. *Proc. VLDB Endow.* 13, 10 (June 2020), 1751–1764. https://doi.org/10.14778/3401960.3401971

[11] Duanbing Chen, Linyuan Lü, Ming-Sheng Shang, Yi-Cheng Zhang, and Tao Zhou. 2012. Identifying influential nodes in complex networks. *Physica A: Statistical Mechanics and its Applications* 391, 4 (2012), 1777–1787. https://doi.org/10.1016/j.physa.2011.09.017

[12] Jonathan Cohen. 2008. *Trusses: Cohesive Subgraphs for Social Network Analysis.* Technical Report. National Security Agency.

[13] Wanyun Cui, Yanghua Xiao, Haixun Wang, and Wei Wang. 2014. Local search of communities in large graphs. In *International Conference on Management of Data, SIGMOD 2014, Snowbird, UT, USA, June 22-27, 2014*, Curtis E. Dyreson, Feifei Li, and M. Tamer Özsu (Eds.). ACM, 991–1002. https://doi.org/10.1145/2588555.2612179

[14] Herbert Edelsbrunner and John Harer. 2010. *Computational Topology: An Introduction.* American Mathematical Society.

[15] David Eppstein, Maarten Löffler, and Darren Strash. 2010. Listing All Maximal Cliques in Sparse Graphs in Near-Optimal Time. In *Proc. International Symposium on Algorithms and Computation (ISAAC)*. 403–414.

[16] Ernesto Estrada and Juan A Rodriguez-Velazquez. 2005. Complex networks as hypergraphs. *arXiv preprint physics/0505137* (2005).

[17] Santo Fortunato. 2010. Community detection in graphs. *Physics Reports* 486, 3 (2010), 75–174. https://doi.org/10.1016/j.physrep.2009.11.002

[18] Christos Giatsidis, Dimitrios M. Thilikos, and Michalis Vazirgiannis. 2011. Evaluating Cooperation in Communities with the k-Core Structure. In *International Conference on Advances in Social Networks Analysis and Mining, ASONAM 2011, Kaohsiung, Taiwan, 25-27 July 2011*. IEEE Computer Society, 87–93. https://doi.org/10.1109/ASONAM.2011.65

[19] Aristides Gionis and Charalampos E. Tsourakakis. 2015. Dense Subgraph Discovery: KDD 2015 tutorial. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Sydney, NSW, Australia, August 10-13, 2015*, Longbing Cao, Chengqi Zhang, Thorsten Joachims, Geoffrey I. Webb, Dragos D. Margineantu, and Graham Williams (Eds.). ACM, 2313–2314. https://doi.org/10.1145/2783258.2789987

[20] M. Girvan and M. E. J. Newman. 2002. Community structure in social and biological networks. *Proceedings of the National Academy of Sciences* 99, 12 (2002), 7821–7826. https://doi.org/10.1073/pnas.122653799 arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.122653799

[21] J E Hirsch. 2005. An index to quantify an individual's scientific research output. *Proc Natl Acad Sci U S A* 102, 46 (2005), 16569–16572.

[22] Xin Huang, Hong Cheng, Lu Qin, Wentao Tian, and Jeffrey Xu Yu. 2014. Querying k-truss community in large and dynamic graphs. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data* (Snowbird, Utah, USA) *(SIGMOD '14)*. Association for Computing Machinery, New York, NY, USA, 1311–1322. https://doi.org/10.1145/2588555.2610495

[23] Shweta Jain and C. Seshadhri. 2020. The Power of Pivoting for Exact Clique Counting. In *Proceedings of the 13th International Conference on Web Search and Data Mining* (Houston, TX, USA) *(WSDM '20)*. Association for Computing Machinery, New York, NY, USA, 268–276. https://doi.org/10.1145/3336191.3371839

[24] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Washington, D.C.) *(KDD '03)*. Association for Computing Machinery, New York, NY, USA, 137–146. https://doi.org/10.1145/956750.956769

[25] Wissam Khaouid, Marina Barsky, Venkatesh Srinivasan, and Alex Thomo. 2015. K-core decomposition of large networks on a single PC. *Proc. VLDB Endow.* 9, 1 (2015), 13–23. https://doi.org/10.14778/2850469.2850471

[26] Dahee Kim, Junghoon Kim, Sungsu Lim, and Hyun Ji Jeong. 2023. Exploring Cohesive Subgraphs in Hypergraphs: The (k,g)-Core Approach. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management* (Birmingham, United Kingdom) *(CIKM '23)*. Association for Computing Machinery, New York, NY, USA, 4013–4017. https://doi.org/10.1145/3583780.3615275

[27] Maksim Kitsak, Lazaros K Gallos, Shlomo Havlin, Fredrik Liljeros, Lev Muchnik, H Eugene Stanley, and Hernán A Makse. 2010. Identification of influential spreaders in complex networks. *Nature physics* 6, 11 (2010), 888–893.

[28] Steffen Klamt, Utz-Uwe Haus, and Fabian Theis. 2009. Hypergraphs and Cellular Networks. *PLOS Computational Biology* 5, 5 (05 2009), 1–6. https://doi.org/10.1371/journal.pcbi.1000385

[29] Jongshin Lee, Kwang-Il Goh, Deok-Sun Lee, and B. Kahng. 2023. (k,q)-core decomposition of hypergraphs. *Chaos, Solitons & Fractals* 173 (2023), 113645. https://doi.org/10.1016/j.chaos.2023.113645

[30] Linyuan Lü, Duanbing Chen, Xiao-Long Ren, Qian-Ming Zhang, Yi-Cheng Zhang, and Tao Zhou. 2016. Vital nodes identification in complex networks. *Physics Reports* 650 (2016), 1–63. https://doi.org/10.1016/j.physrep.2016.06.007

[31] Linyuan Lu, Tao Zhou, Qian-Ming Zhang, and H. Eugene Stanley. 2016. The h-Index of a Network Node and Its Relation to Degree and Coreness. *Nature Communications* 7 (2016), 10168.

[32] Mingsong Mao, Jie Lu, Jialin Han, and Guangquan Zhang. 2019. Multiobjective e-commerce recommendations based on hypergraph ranking. *Information Sciences* 471 (2019), 269–287. https://doi.org/10.1016/j.ins.2018.07.029

[33] Sergei Maslov and Kim Sneppen. 2002. Specificity and stability in topology of protein networks. *Science* 296, 5569 (2002), 910–913.

[34] Juan Manuel Matalobos Veiga, Regino Criado, Miguel Romance Del Rio, Sergio Iglesias Perez, Alberto Partida Rodriguez, and Karan Kabbur Hanumanthappa Manjunatha. 2024. A Hypergraph-based Model for Cyberincident-related Data Analysis. In *Proceedings of the 2024 European Interdisciplinary Cybersecurity Conference* (Xanthi, Greece) *(EICC '24)*. Association for Computing Machinery, New York, NY, USA, 161–162. https://doi.org/10.1145/3655693.3661300

[35] Alberto Montresor, Francesco De Pellegrini, and Daniele Miorandi. 2013. Distributed k-Core Decomposition. *IEEE Transactions on Parallel and Distributed Systems* 24, 2 (2013), 288–300. https://doi.org/10.1109/TPDS.2012.124

[36] Ahmet Erdem Saríyüce, Buğra Gedik, Gabriela Jacques-Silva, Kun-Lung Wu, and Ümit V. Çatalyürek. 2013. Streaming algorithms for k-core decomposition. *Proc. VLDB Endow.* 6, 6 (2013), 433–444. https://doi.org/10.14778/2536336.2536344

[37] Ahmet Erdem Sariyüce and Ali Pinar. 2016. Fast hierarchy construction for dense subgraphs. *Proc. VLDB Endow.* 10, 3 (Nov. 2016), 97–108. https://doi.org/10.14778/3021924.3021927

[38] Ahmet Erdem Sariyüce, C. Seshadhri, and Ali Pinar. 2018. Local algorithms for hierarchical dense subgraph discovery. *Proc. VLDB Endow.* 12, 1 (Sept. 2018), 43–56. https://doi.org/10.14778/3275536.3275540

[39] Ahmet Erdem Sariyüce, C. Seshadhri, Ali Pinar, and Ümit V. Çatalyürek. 2015. Finding the Hierarchy of Dense Subgraphs Using Nucleus Decompositions. In *Proc. International Conference on World Wide Web (WWW)*. 927–937.

[40] Ahmet Erdem Sariyüce, C. Seshadhri, Ali Pinar, and Ümit V. Çatalyürek. 2017. Nucleus Decompositions for Identifying Hierarchy of Dense Subgraphs. *ACM Trans. Web* 11, 3, Article 16 (July 2017), 27 pages. https://doi.org/10.1145/3057742

[41] Stephen B. Seidman. 1983. Network Structure and Minimum Degree. *Social Networks* 5, 3 (1983), 269–287.

[42] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2021. Theoretically and practically efficient parallel nucleus decomposition. *Proc. VLDB Endow.* 15, 3 (Nov. 2021), 583–596. https://doi.org/10.14778/3494124.3494140

[43] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2024. Parallel Algorithms for Hierarchical Nucleus Decomposition. *Proc. ACM Manag. Data* 2, 1, Article 32 (March 2024), 27 pages. https://doi.org/10.1145/3639287

[44] Etsuji Tomita, Akira Tanaka, and Haruhisa Takahashi. 2006. The Worst-Case Time Complexity for Generating All Maximal Cliques and Computational Experiments. *Theoretical Computer Science* 363, 1 (2006), 28–42.

[45] Dong Wen, Lu Qin, Ying Zhang, Xuemin Lin, and Jeffrey Xu Yu. 2016. I/O efficient core graph decomposition at web scale. In *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*. IEEE, 133–144.

[46] Dong Wen, Lu Qin, Ying Zhang, Xuemin Lin, and Jeffrey Xu Yu. 2019. I/O Efficient Core Graph Decomposition: Application to Degeneracy Ordering. *IEEE Trans. Knowl. Data Eng.* 31, 1 (2019), 75–90.

[47] Dong Wen, Bohua Yang, Lu Qin, Ying Zhang, Lijun Chang, and Rong-Hua Li. 2022. Computing K-Cores in Large Uncertain Graphs: An Index-Based Optimal Approach. *IEEE Transactions on Knowledge and Data Engineering* 34, 7 (2022), 3126–3138. https://doi.org/10.1109/TKDE.2020.3023925

[48] Jaewon Yang and Jure Leskovec. 2015. Defining and Evaluating Network Communities Based on Ground-Truth. *Knowledge and Information Systems* 42, 1 (2015), 181–213.

[49] Wenqian Zhang, Zhengyi Yang, Dong Wen, Yi Ding, Wenjie Zhang, and Xuemin Lin. 2026. Nucleus Decomposition Revisited: An Efficient Counting-Based Approach. *Proc. ACM Manag. Data* 4, 3, Article 215 (May 2026), 26 pages. https://doi.org/10.1145/3802092

[50] Wenqian Zhang, Zhengyi Yang, Dong Wen, and Xiaoyang Wang. 2023. Efficient Distributed Core Graph Decomposition. In *2023 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 1023–1031.

[51] Feng Zhao and Anthony K. H. Tung. 2012. Large scale cohesive subgraphs discovery for social network visual analysis. *Proc. VLDB Endow.* 6, 2 (2012), 85–96. https://doi.org/10.14778/2535568.2448942